UNIVERSIDAD DE EXTREMADURA

ESCUELA DE INGENIERÍAS INDUSTRIALES

Diseño de sistema de control de semáforos en intersecciones aisladas

TRABAJO PRESENTADO PARA OPTAR AL TITULO DE

GRADO INGENIERIA ELECTRONICA Y AUTOMATICA (RAMA INDUSTRIAL)

POR

ÁNGEL CANELO CALDERA

BADAJOZ, JULIO DE 2015

TRABAJO FIN DE GRADO

Autor:

ÁNGEL CANELO CALDERA

Director o Directores:

José Vicente Valverde Sánchez

Tribunal de evaluación:

Manuel Calderón Godoy

José Ignacio Suárez Marcelo

Miguel Ángel Domínguez Puertas

ÍNDICE

1 INTRODUCCIÓN 3
1.1 Los semáforos y su historia 3
1.2 Los semáforos actuales 5
1.3 Funcionamiento básico 6
2 LA REGULACIÓN DE SEMÁFOROS 7
2.1 Necesidad de regular intersecciones 7
2.2 Elementos utilizados para el control en una intersección 7
2.2.1 Los reguladores 8
2.2.2 Los detectores 9
2.3 Conceptos necesarios para la regulación de semáforos 11
2.3.1 Determinación de las fases 11
2.3.2 Calculo del tiempo de ciclo y de reparto 11
2.4 Control de semáforos desde sala de control 16
2.5 Funcionamiento de un cruce de semáforos 16
2.5.1 Cruces a tiempos fijos 17
2.5.2 Cruces semiactuados 17
2.5.3 Cruces actuados 17
2.5.4 Comparativa entre los actuados y los no actuados 18
2.6 Sincronización entre intersecciones 19
2.6.1 Sincronización mediante los mismos reguladores 20
2.6.2 Sincronización mediante un equipo diferente a los reguladores 20
3 DISEÑO DEL SISTEMA 21
3.1 Objetivo del trabajo 21
3.2 Planteamiento del problema 21
3.3 Solución al problema 21
3.3.1 Utilización de microcontroladores PIC 22
3.3.2 Utilización del bus UART 22
3.3.3 Comunicación USB 22
4 ESTRUCTURA DEL SISTEMA 24
4.1 Estructura global del sistema 24
4.2 PC 24
4.3 Placa PIC18F47J53 FS USB PIM DEMO BOARD 25
4.4 Microcontroladores PIC16 de cada semáforo 27
5 RECREACIÓN DE UNA INTERSECCIÓN PARA REPRESENTAR EL FUNCIONAMIENTO DEL SISTEMA 29
5.1 Diseño de las PCB’s para los PIC16 29

5.2 Fabricación de las PCB's 30
5.3 Puesta en marcha 31
6 MATERIALES Y HERRAMIENTAS UTILIZADAS 33
6.1 PIC18F47J53 FS USB PIM DEMO BOARD 33
6.2 PIC16F1704-I/P 33
6.3 Comunicación USB (Universal Serial Bus) 34
6.4 Comunicación UART 36
6.5 Diodos LED 36
6.6 Resistencias de 1kΩ y 10kΩ 37
6.7 Baquelita positiva a una cara para fabricación de PCB's 38
6.8 MPLAB IDE v8.92 39
6.9 Microchip Solutions 40
6.10 Microsoft Visual Studio 2010 40
6.11 CadSoft Eagle 40
6.12 PICkit 3 41
7 CONCLUSIONES Y FUTURAS AMPLIACIONES 43
8 BIBLIOGRAFÍA 44
9 ANEXO 45
9.1 Programación PIC18 45
9.2 Programación PIC16 97
9.2.1 Semáforo de 3 colores 97
9.2.2 Semáforo de peatones 103
9.2.3 Semáforo de flecha 109
9.2.4 Inicializaciones.inc 115
9.2.5 Interrupciones.inc 117
9.2.6 Rutinas.inc 122
9.2.7 Banco.inc 123
9.2.8 Interrupciones.inc en semáforo de peatones 124
9.3 Programación aplicación de escritorio (Visual Studio 2010) 129

1 INTRODUCCIÓN

1.1 Los semáforos y su historia

Los semáforos son dispositivos de señales que son empleados en el control/regulación de tráfico de forma automática de vehículos y peatones en las intersecciones, cruces y en cualquier otro lugar que necesite de una herramienta para dar preferencias secuenciales a los distintos participantes de la circulación.

El auge de las señales de tráfico está ligado al rápido aumento del tráfico automovilístico. El tráfico urbano brotó enormemente después de que Henry Ford introdujo el modelo T en 1908.

El 9 de diciembre de 1868 se instaló el primer semáforo en Londres. Fue diseñado por el ingeniero ferroviario John Peake Knight, quién se basó en las señales ferroviarias de la época. El primer semáforo fue muy diferente a los actuales, con dos brazos que se levantaban para indicar el sentido en que tenía que detenerse, además utilizaba lámparas de gas de colores rojo y verde para su uso nocturno. Sin embargo, este primer semáforo era manual por lo que requería que un policía lo controlase todo el tiempo.

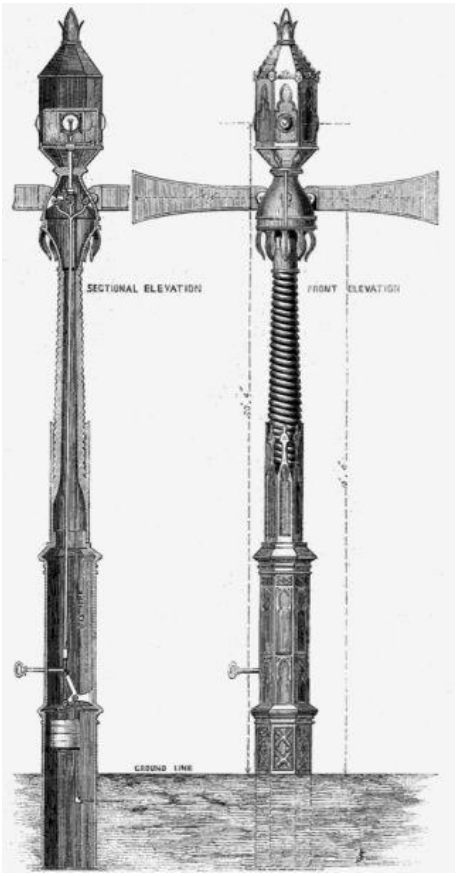


Figura 1.1 Diseño del semáforo de J. P. Knight.

En 1910, Ernest Sirrine mejoró el semáforo volviéndolo automático. El sistema de Sirrine usó las palabras no iluminados «stop » (detenerse) y «proceed » (proceder). Fue el 5 de agosto de 1914 en Cleveland, en el estado norteamericano de Ohio, donde se iluminó el primer semáforo de la historia tal y como lo conocemos hoy en día. Parece ser que su creación fue compartida, entre

Lester Farnsworth Wire, encargado de la división de tráfico del Departamento de Policía de Salt Lake City, Utah, y Garrett Morgan, hijo de un antiguo esclavo que también inventó la máscara de gas.

Avances en la tecnología de la ingeniería eléctrica eflectuaron mejoras en la señalización posible. Los ingenieros habían desarrollado programadores automáticos para sistemas de comunicaciones militares durante la Primera Guerra Mundial. El primer semáforo automático que utilizaba luces rojas y verdes eléctricas fue patentado por William Ghiglieri en San Francisco, California en 1917. Su diseño también incluía un modo manual.



Figura 1.2 Semáforo en Estocolmo, 1953.

Los primeros semáforos de las calles copiaron el código de colores del sistema ferroviario: el color rojo significaba parar, el verde significaba precaución y añadieron el blanco, que significaba avanzar. Pero este último creó muchos problemas de seguridad y decidieron hacerse cambios. Se mantuvo el rojo (asociado a la sangre y al peligro) para detenerse. El verde, fácilmente visible por la noche (ligado a la salud y la naturaleza) fue el designado para indicar que se podía pasar. El amarillo se eligió porque se ve fácilmente desde lejos (incluso de noche) y se relacionaba con el sol, un elemento que se consideró que podía calmar a los conductores.

En 1920, William Potts, inventor de varios semáforos para ese momento, añade la luz amarilla al semáforo. Para ese momento los semáforos ya se habían expandido alrededor del mundo, pero, a diferencia de los demás de dos etapas, el de tres etapas permitía advertir de una mejor manera al conductor sobre el inminente cambio a la luz roja. Potts era un oficial de policía de Detroit, por lo

que al igual que Wire nunca patentó su invención. Garrett Morgan en 1923 fue la primera persona en lograr la patente por un semáforo de tres etapas eléctrico en 1923.

Continuando el modelo americano, fue Siemens quien diseñó para el cruce de la Postdamer Platz de Berlín su primer semáforo eléctrico, una revolución para la Europa de entre guerras. Poco después, en 1933 Copenhague estrenó un diseño que incluía por primera vez la luz peatonal. Casi veinte años después, Nueva York fue la primera ciudad con semáforos que indicaban a los peatones cuando esperar o cruzar. En 1961 aparecería un nuevo elemento, el muñeco que representa al peatón, diseñado por el psicólogo Karl Peglau en la Alemania Oriental.

1.2 Los semáforos actuales

Actualmente existen diferentes tipos de semáforo. Algunos funcionan por tiempo (se configuran para que la luz cambien en un perídodo determinado), pero cada vez más se opta por centros de control que, mediante algoritmos, regulan el tráfico, permitiendo que la circulación sea más fluida.

Los semáforos han ido evolucionando con el paso del tiempo y actualmente y debido a su rentabilidad, se están utilizando lámparas LED para la señalización luminosa, puesto que las lámparas LED utilizan solo el 10 % de la energía consumida por las lámparas incandescentes, tienen una vida estimada 50 veces mayor, y por tanto generan importantes ahorros de energía y mantenimiento, satisfaciendo el objetivo de conseguir una mayor fiabilidad y seguridad pública.

Aunque los ledes ofrece multitud de ventajas respecto a las bombillas tradicionales uno de sus mayores inconvenientes es que no soportan bien los cambios bruscos de energía, que es lo que ocurre cuando se encienden o se apagan cada una de las luces del semáforo, ya que además cada luz debe apagarse rápidamente para no provocar confusión con el resto de las luces, lo que provoca que algunos diodos se fundan.



Figura 1.3 Semáforo LED.

1.3 Funcionamiento básico

El funcionamiento básico de los semáforos y el más extendido es el de tres colores:

Rojo (1), para detenerse inmediatamente. En algunos países, si el rojo está parpadeando, actúa como una señal de Alto/Pare/Stop.

Verde (2), para avanzar, puesto que no hay obstáculos.

Amarillo o Ámbar (3) precaución/ceda el paso.

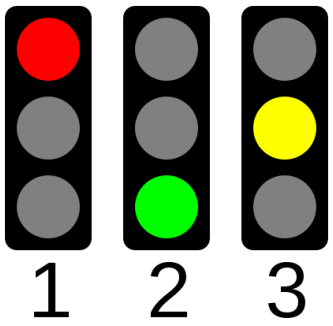


Figura 1.4 Semáforo de 3 etapas.

Pero realmente hay una gran variedad de ellos y más específicos, como pueden ser los semáforos para los autobuses, los peatonales, para las bicicletas, para los cambios de carril, etc.

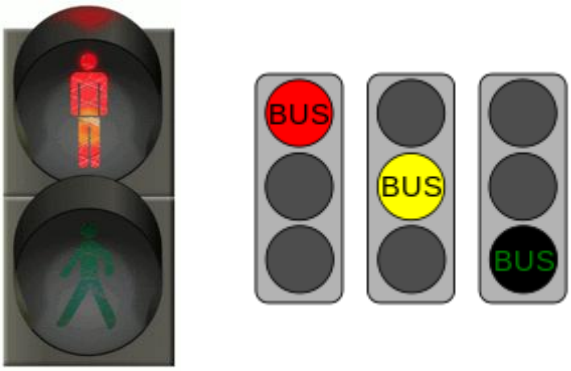


Figura 1.5 Otros tipos de semáforos

2 LA REGULACIÓN DE SEMÁFOROS

2.1 Necesidad de regular intersecciones

Cuando dos o más vías, de uno o dos sentidos de circulación, se cruzan a un mismo nivel y la intensidad de tráfico es más grande que la que se puede admitir con una regulación de preferencia de paso, la utilización de semáforos se ha mostrado como un medio seguro, económico y eficaz para regular el tráfico. Como inconveniente se producen detenciones y por tanto demora a los vehículos que acceden a la intersección.

Funcionalmente, los semáforos, permiten el paso alternativo de los vehículos desde los distintos accesos que confluyen a la intersección durante un tiempo determinado. Este paso es concedido ordenadamente y se repite en el tiempo de forma cíclica. Cuando los movimientos son "paralelos" o "divergentes", pueden realizarse al mismo tiempo aunque sean de distintos accesos. No pueden realizarse al mismo tiempo cuando son "convergentes" o se "cruzan".

En una intersección puede regularse exclusivamente la circulación de vehículos, pero, de una forma más general y principalmente en zona urbana, suele regularse la circulación de vehículos y peatones.

2.2 Elementos utilizados para el control en una intersección

Son necesarios dos elementos para la regulación de los semáforos, los reguladores y los detectores.

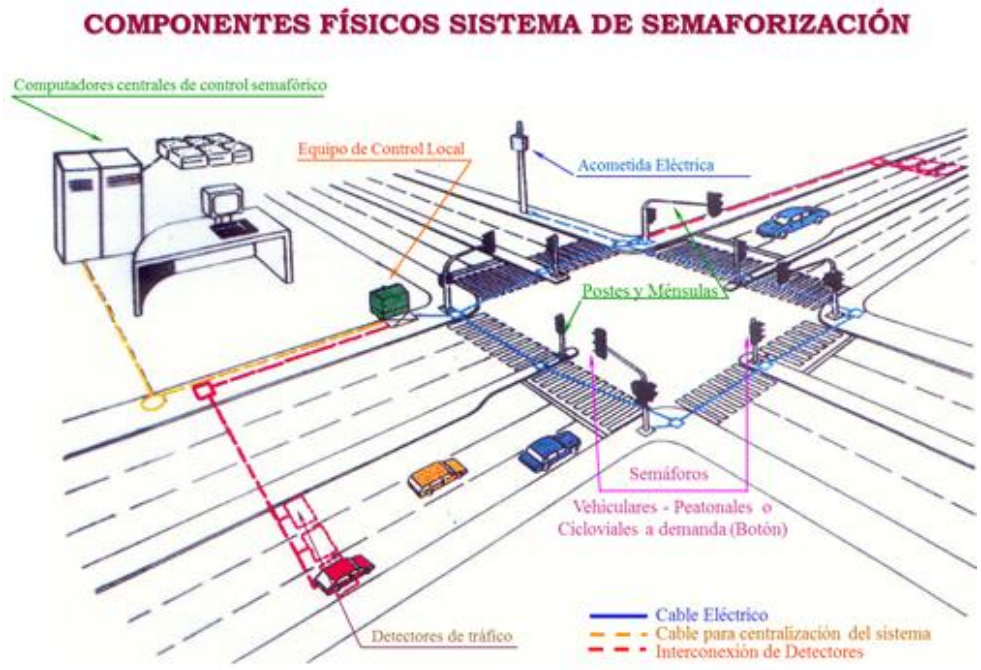


Figura 2.1 Elementos de un cruce semafórico

2.2.1 Los reguladores

Para controlar el funcionamiento de los semáforos instalados en un cruce o en dos o más muy próximos, se utiliza un aparato denominado regulador, que sirve para ordenar los cambios de luces a los semáforos instalados en una intersección.



Figura 2.2 Regulador en Barcelona

Los reguladores pueden ser electromecánicos, electrónicos o de microprocesador. Se comprende que los semáforos van conectados al equipo regulador a través de los cables eléctricos necesarios para su funcionamiento y control. Por tanto, en la Regulación Automática del Tráfico con Semáforos intervienen dos partes distintas y complementarias a la vez:

- Primero, la ingeniería de tráfico que es quien determina teóricamente la manera en que los semáforos han de funcionar, para conseguir un resultado práctico sobre la circulación de vehículos y personas.

- Segundo, la ingeniería eléctrica, electrónica e informática, que hacen realidad el plan teórico, encendiendo y apagando las lámparas de los semáforos con precisión en el tiempo y realizando todas las funciones complementarias necesarias.

El regulador tiene por función primaria el encendido y apagado de todas las lámparas de los semáforos, aunque también puede controlar el encendido y apagado de señales de información variable y de señales ocultas.

El encendido y apagado de las lámparas se realiza de acuerdo con el PLAN DE TRAFICO que determina los movimientos de tráfico autorizados que se realizan en la intersección, su orden y el tiempo de duración que tiene cada uno de ellos.

Los reguladores pueden ser autónomos, sincronizables o centralizables. En el primer caso, funcionan aislados sin enviar ni recibir señales a ningún otro regulador. En el segundo han de ser capaces de enviar y de recibir señales de sincronismo. En el tercer caso han de estar equipados con el interface y el protocolo de comunicaciones adecuado para recibir y enviar las señales de control y de información del sistema.

Los reguladores pueden ser a tiempos fijos, actuados o semiactuados, si bien los modernos reguladores basados en el microprocesador, suelen funcionar indistintamente en cualquiera de las tres formas.

El regulador deberá constar, al menos, de las siguientes partes:

- Fuente de alimentación con fusible de entrada.
- Módulo central con los elementos de cálculo y control, memorias, etc.
- Módulos de salida, en un número variable de ellos, que incorpora los elementos encargados de suministrar la potencia a las lámparas.
- Módulo de comunicaciones para comunicarse con el sistema de C.T. u otros reguladores.

Los reguladores van instalados dentro de armarios metálicos de doble chapa, preparados para estar a la intemperie y, en función del modelo, pueden llevar ventilador, luz, toma de tensión, etc. así como la regleta de conexión de los cables de semáforo.

2.2.2 Los detectores

Se denomina detector a todo dispositivo que registra y transmite cualquier información referente a una determinada característica o parámetro del tráfico (intensidad, velocidad, densidad, ocupación) de la vía o lo que es lo mismo, cualquier dispositivo capaz de registrar y transmitir los cambios que se producen, o los valores que se alcanzan, en un determinado parámetro del tráfico o de la vía.

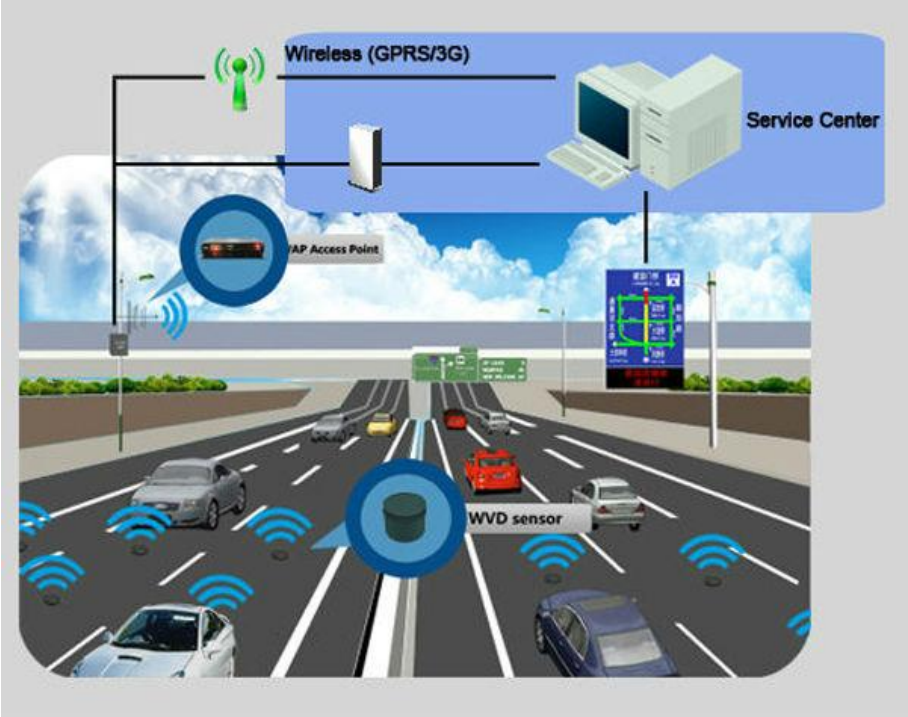


Figura 2.3 Matriz de detectores para regulación

Tradicionalmente se han utilizado las espiras insertadas en el pavimento, fácilmente destruibles por los equipos de mantenimiento en campañas de fresado. Estas espiras pueden ser sustituidas por equipos basados en visión artificial, que proporcionan los parámetros anteriormente citados, pero cuyo mayor inconveniente es el coste.

Son muy numerosas las variantes que pueden encontrarse en el mercado, las más extendidas son las siguientes.

2.2.2.1 Detectores de presión

Consisten esencialmente en una plancha de caucho en cuyo interior se sitúan dos láminas metálicas, muy cerca una de la otra, que establecen contacto cuando pasa una carga sobre la plancha. Todo ello va colocado en la parte superior de una plataforma de hormigón o metálica que se empotra en el pavimento.

Puede conseguirse una detección direccional si en vez de las dos láminas metálicas se ponen cuatro, enfrentadas dos a dos.

2.2.2.2 Detectores de lazo

Llamados así por estar constituidos por un bucle, lazo o espira metálica que se empotra en el pavimento y se conecta a un amplificador. Detecta la variación de inductancia que se produce cuando pasa un vehículo por encima de la espira.

2.2.2.3 Detectores de radar y similares

Constan de un aparato emisor y otro receptor de ondas electromagnéticas y generalmente se suspenden sobre la vía o se colocan lateralmente a ella.

Están basados en el efecto Dopler que se produce al chocar un tren de ondas contra algo que se mueve.

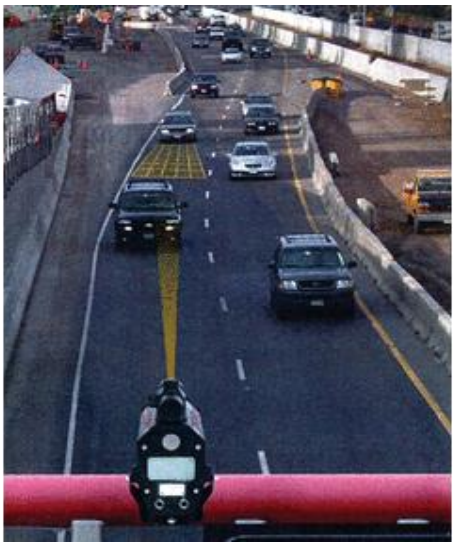


Figura 2.4 Detector tipo cámara

Los detectores basados en la propagación de las ondas, son muy similares en su funcionamiento a los de radar. Entre ellos se pueden destacar especialmente los ultrasónicos y también los fotoeléctricos y los neumáticos. También se pueden utilizar detectores basados en los rayos Laser.

2.3 Conceptos necesarios para la regulación de semáforos

Se llama ciclo al tiempo transcurrido desde el cambio de un grupo semafórico hasta la repetición de dicha situación después de realizarse una secuencia de maniobra completa en los semáforos conectados a un mismo regulador.

Se denomina fase a cada una de las divisiones del ciclo durante la cual la configuración de colores de todos los grupos semafóricos permanece invariable.

El término despeje se refiere al tiempo necesario para que los vehículos que han accedido a la intersección por uno de los ramales o calles, salgan de la zona de intersección y la dejen totalmente libre para que entren en la misma los vehículos de otros ramales sin que exista peligro de colisión de los mismos.

Por último, reparto del ciclo es la división del tiempo del ciclo entre cada una de las fases que integran la estructura de regulación propia del plan de tráfico operativo.

2.3.1 Determinación de las fases

La determinación de las fases, es decir de los movimientos que pueden darse simultáneamente, no puede sujetarse a reglas fijas, sino que dependerán, en general, de las características del tráfico y del trazado de la intersección.

Los criterios que deben presidir el estudio de las fases son los siguientes:

1. El número de fases debe ser lo menor posible. Con ello se reducen al mínimo los tiempos perdidos en cada ciclo.

2. El número de movimientos simultáneos--sin conflicto entre sí debe ser máximo.

3. El recorrido dentro de la intersección se procurará que sea lo más corto posible. Así se logran unos tiempos de despeje más cortos.

4. Cuando un ciclo se divide en más de dos fases es necesario considerar el orden en que se producen, ya que ello influye en la seguridad y rendimiento de la intersección.

5. Cuando durante la fase de peatones, tanto el número de éstos como el de vehículos que tratan de salir de la intersección no serán muy elevados, puede permitirse el paso de ambos asignando la preferencia a los peatones. Si uno de los dos movimientos es relativamente importante, hay que considerar unas fases que los separen totalmente, ya que en caso contrario podría resultar peligroso para los peatones o producirse colas que bloqueen la intersección.

2.3.2 Calculo del tiempo de ciclo y de reparto

2.3.2.1 Reparto de tiempos verdes

Independientemente de lo que resulte de los cálculos, la duración del ciclo tiene que estar forzosamente comprendida entre los límites que fija la psicología del conductor. La práctica indica que ciclos menores de 35 segundos o mayores de 120 se acomodan difícilmente a la mentalidad del usuario de la vía pública

En gran parte de los casos, la proximidad entre intersecciones obliga a que se adopte una misma duración de ciclo. Cuando las distancias entre intersecciones son grandes, es posible elegir ciclos distintos, pues se produce una dispersión de los vehículos que circulaban agrupados.

El caso más sencillo es aquél en que se pretende repartir un ciclo de una duración dada entre dos calles con una intensidad de tráfico conocida; para resolver el caso más desfavorable, se toma la que corresponde a los 15 minutos punta. La primera aproximación, que en muchos casos es suficiente, consiste en repartir el ciclo proporcionalmente a las intensidades máximas por carril de cada calle.

En este caso se tendría que:

$$\frac{Ta}{Tb} = \frac{Ia}{Ib} \tag{2.1}$$

$$\mathrm{Ta} + \mathrm{Tb} = \mathrm{C} \tag{2.2}$$

Donde Ia, Ib = Intensidades máximas por carril en las calles A y B respectivamente.

Ta, Tb = Tiempos de verde correspondientes a cada calle.

C = Duración del ciclo en segundos.

Como casi siempre el tráfico tiene unas características diferentes en cada calle, conviene tener también en cuenta el intervalo más frecuente con que se suceden los vehículos en cada vía. En ese caso se hacen proporcionales los tiempos verdes al producto de las intensidades por carril y el intervalo más frecuente de los vehículos en cada calle:

$$\frac{Ta}{Tb} = \frac{Ia \times Ea}{Ib \times Eb} \tag{2.3}$$

Donde Ea y Eb son los intervalos más frecuentes observados en las calles A y B, respectivamente.

El reparto así obtenido no se puede adoptar sin más análisis. En muchos casos hay que tener en cuenta el tiempo mínimo necesario para que los peatones atraviesen la calzada. Si ese tiempo mínimo es mayor que el tiempo de paso asignado al movimiento que se realiza simultáneamente con el paso de peatones, habrá que modificar el reparto o el ciclo hasta que los peatones tengan tiempo suficiente para cruzar.

2.3.2.2 Duración del tiempo en amarillo

La utilización de la luz amarilla entre la verde y la roja se debe a que no es posible detener instantáneamente un vehículo. Su finalidad es la de avisar al conductor que va a aparecer la luz roja y que, por tanto, debe decidir si tiene tiempo para pasar antes de que se encienda o si, por el contrario, no lo tiene y ha de frenar.

En muchos tratados de ingeniería de tráfico se hace el cálculo de la duración del amarillo basándose en los dos supuestos siguientes:

1. El tiempo de amarillo será igual o superior al requerido para frenar antes de la línea de detención.

2. Si se ha entrado en la intersección, dará tiempo a atravesarla antes de que se encienda la luz roja.

Para que en todo momento se cumplan ambos supuestos habrá de tomarse siempre el mayor de los valores que resulte de calcular el tiempo de amarillo con ambos criterios.

Para el primer supuesto tenemos:

$$t1 = R + 2d/v \qquad (2.4)$$

En donde t1 es el tiempo de frenado, R el tiempo de percepción y reacción, v la velocidad de un vehículo representativo y d la distancia de frenado; se parte de la base, no exacta pero aceptable, de que la deceleración es constante.

Si la anchura de la intersección es a, el tiempo t2 necesario para reaccionar, llegar a la intersección y atravesarla a la velocidad v será:

$$t2 = R + d/v + a/v \qquad (2.5)$$

Como la distancia de frenado es proporcional al cuadrado de la velocidad, cuando ésta sea baja, el tiempo de amarillo vendrá dado por la anchura de la intersección. Tomando velocidades cada vez mayores se llegará a un punto en que la distancia determinante sea la de frenado.

A pesar de que los criterios antes indicados parecen muy razonables, como con la aplicación de las fórmulas (2.4) y (2.5) se llega generalmente a tiempos de amarillo muy largos, la mayoría de los autores recomiendan reducirlo a 3 ó 4 segundos, ya que la práctica indica que los valores teóricos generalmente obtenidos son menos eficaces que los más reducidos aconsejados por la experiencia. La razón de esta contradicción se encuentra probablemente en el hecho de que el cálculo teórico no tiene exactamente en cuenta lo que ocurre en la realidad.

La introducción de los tiempos de “todo rojo” hace innecesario incluir en el amarillo el tiempo de despeje, con lo cual el único valor que cuenta es el del tiempo de frenado. Supóngase un conductor que ve encenderse la luz amarilla. Si su distancia a la línea de parada es inferior a la mínima de frenado deberá proseguir. Si es igual o superior, podrá parar y el tiempo mínimo ta que necesitará para hacerlo será:

$$ta = 2d/v + R \qquad (2.6)$$

Si a pesar de poder frenar a tiempo decide no hacerlo y continúa con la velocidad que llevaba, tardará tb segundos en alcanzar la línea de parada y se tendrá que:

$$tb = d/v + R \qquad (2.7)$$

Y como ta-tb= d/v , será siempre ta > tb, o lo que es lo mismo, si la duración del amarillo es igual al tiempo de frenado, como la teoría tradicional aconseja, habrá un período de tiempo (d/v) en que el conductor podrá elegir entre detenerse o seguir sin obedecer a la luz roja. El resultado de esa posibilidad de elección será que una inmensa mayoría optará por seguir, ya que es lo que menos esfuerzo requiere y lo que más tiempo ahorra.

Cuanto menor sea el intervalo en que pueden tomarse decisiones contrapuestas, es mayor la seguridad de funcionamiento. Como lo que realmente importa es la distancia de frenado y no el tiempo, la luz amarilla debe encenderse de forma que si en ese momento hay un vehículo situado

a una distancia superior a la de frenado tenga que parar forzosamente y si está a una distancia menor, mantenga su velocidad y pase.

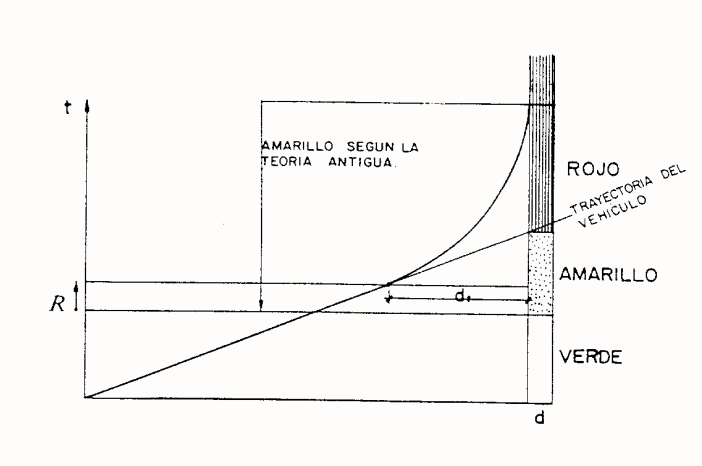


Figura 2.4 Duración de la luz amarilla

Para ello el tiempo en amarillo será el que se tarde en recorrer la distancia d, a la velocidad v. es decir:

$$t = d1/v + R \qquad (2.8)$$

Puesto que la distancia de frenado depende a su vez de la velocidad y está ligada a ella por una relación de tipo:

$$d1 = v2/K \qquad (2.9)$$

En la que K es una constante que tiene en cuenta el coeficiente de rozamiento.

Un valor perfectamente posible de la constante podría ser K = 10, con lo que:

$$t = \frac{v}{10} + R \qquad (2.10)$$

Que para una velocidad de 60 km./h. (16,6 m./seg.) y un tiempo de reacción de 1,5 segundos da una duración del amarillo de 3,16 segundos, que está de acuerdo con la práctica habitual.

La tendencia general es llegar a una duración del amarillo uniforme, con lo cual los conductores reaccionarán siempre de la misma forma y las únicas variables que intervendrán en su decisión serán la distancia a que se encuentran del semáforo y la velocidad a que circulan.

Solamente en vías de gran velocidad puede recurrirse a tiempos de amarillo un poco más largos, aunque en estas vías no abundan los semáforos a no ser en puntos donde cambia el carácter de la calle o carretera, que suelen ser muy peligrosos y conviene preseñalizar adecuadamente.

La duración del amarillo, es un factor que hay que tener en cuenta, pues en los ciclos cortos puede llegar a representar un porcentaje apreciable del tiempo total. Con un tiempo amarillo de 3 segundos y un ciclo de 30, el semáforo está el 10 por 100 del tiempo en amarillo, mientras que si el ciclo es de 90 segundos sólo lo está el 3,3 por 100.

2.3.2.3 Longitud del ciclo

Puesto que el tiempo de salida de un vehículo es mayor que el intervalo medio entre varios en movimiento, a mayor cantidad de ciclos por hora es decir, a menor ciclo se efectuarán mayor número de arrancadas y, por consiguiente, se acumularán mayores pérdidas de tiempo.

A pesar de lo anterior, casi siempre se procura obtener un ciclo lo más corto posible, ya que con ello se suelen reducir los tiempos de espera, aunque la capacidad disminuya con la duración del ciclo.

Un método bastante exacto para el cálculo de la longitud del ciclo es el siguiente:

Sea Ii la intensidad que se registra en cada uno de los 1,2,....i, ..n accesos a la intersección y Ci la capacidad de los mismos.

Para cada uno de estos movimientos se necesitarían como mínimo:

$$\frac{\mathrm{Ii}}{\mathrm{Ci}} \times C \text{ segundos} \qquad (2.11)$$

Siendo C la longitud del ciclo también en segundos. Como hay que tener en cuenta que en cada ciclo se pierde un tiempo Pi en reaccionar, arrancar, frenar y despejar la intersección, el tiempo que habrá que asignar a cada movimiento será:

$$\mathrm{Tn} = \mathrm{Pn} + \frac{\mathrm{In}}{\mathrm{Cn}} \times C \qquad (2.12)$$

Y como la suma de todos los tiempos asignados a cada movimiento (o conjunto de movimientos simultáneos) es igual al ciclo:

$$\mathrm{C} = \sum \mathrm{Pi} + C \times \sum \frac{\mathrm{Ii}}{\mathrm{Ci}} \qquad (2.13)$$

Si llamamos Pt, a la suma de todos los tiempos perdidos e Y a la suma de todas las relaciones intensidad capacidad, se podrá escribir:

$$\mathrm{C} = \mathrm{Pt} + \mathrm{CY} \qquad (2.14)$$

O lo que es lo mismo:

$$\mathrm{Cmin} = \mathrm{Pt} / 1 - \mathrm{Y} \qquad (2.15)$$

Que es, medido en segundos. el ciclo más corto con que puede funcionar una intersección regulada por semáforos. Habrá que tratar, por tanto, de que el ciclo esté algo por encima del valor dado por la fórmula anterior ya que de lo contrario, la congestión aumentaría rápidamente al producirse cualquier incidencia.

A este respecto, el ciclo que produce las menores demoras en el conjunto de los usuarios, llamado también ciclo óptimo, es:

$$\mathrm{Copt} = \frac{1.5 \times \sum Pi + 5}{1 - \sum \frac{Ii}{Ci}} = \frac{1.5 \times Pt + 5}{1 - Y} \qquad (2.16)$$

Cuando una misma fase da lugar a dos o más movimientos, cada uno de ellos pre-sentará una relación I/C distinta. El valor que debe tomarse para el cálculo, como representativo de esa fase será el del movimiento en que la relación intensidad-capacidad sea más alta.

2.4 Control de semáforos desde sala de control

En una sala o centro de gestión de tráfico se centraliza toda la información que puede obtenerse del estado del tráfico por varios medios; por los detectores instalados en la calzada, por medio de cámaras de televisión instaladas en grandes mástiles, e incluso por indicación de agentes de policía. Esta información una vez procesada por ordenadores permite modificar los ciclos implantados en los reguladores.

La información enviada llega a una central de zona que a su vez la distribuye a los reguladores que de ella cuelgan. Esta información una vez recibida en cada regulador modificará los ciclos para producir el efecto que desde la sala ha ordenado el operador.

Una de las funciones más importantes de la regulación semafórica en las grandes ciudades es la disminución de las paradas en los cruces semafóricos con el fin de disminuir los tiempos de recorrido y actualmente, disminuir la contaminación atmosférica y el gasto energético.

Debe conseguirse una coordinación de los semáforos instalados en una vía de tal forma que gracias a un desfase en la aparición de los verdes sucesivos nos permita llegar al siguiente semáforo en verde. Lo que se llama “onda verde”.

2.5 Funcionamiento de un cruce de semáforos

Los semáforos de una intersección pueden funcionar de dos formas diferentes:

- A tiempos fijos.
- Actuados por el tráfico.

Se dice que un cruce semafórico funciona a tiempos fijos, cuando el tiempo que tiene cada acceso se repite invariablemente.

En los cruces actuados, el derecho de paso viene determinado por la demanda del tráfico en cada uno de los accesos por medio de detectores convenientemente instalados o, bien, a través de pulsadores cuando la demanda la realizan los peatones.

En la regulación actuada hay que distinguir dos modalidades:

- La actuación total.
- La semiactuación.

En la actuación total no existe ningún acceso preferente con respecto a las demás y los cambios se realizan según la demanda producida en los accesos, sin más restricción que el orden de la demanda y un tiempo máximo por acceso. La actuación total es poco utilizada en la práctica.

En los cruces semiactuados uno de los accesos se considera principal y el resto secundarios. El acceso principal está normalmente en verde y el resto en rojo. Cuando en uno de los accesos secundarios se detecta una demanda, se corta el verde del acceso principal y se da verde al acceso que tiene demanda. Una vez servido éste, el acceso principal se pone nuevamente en verde. Cuando hay demanda en más de un acceso secundario, se concede derecho de paso de acuerdo con un orden establecido en el regulador, retornando siempre al verde del acceso principal. En caso de demanda continuada el regulador semiactuado funcionaría exteriormente como un regulador de tiempos fijos.

2.5.1 Cruces a tiempos fijos

Los reguladores a tiempos fijos no hacen sino cumplir monótonamente lo que previamente se les ha ordenado, sin depender para nada de las variaciones en intensidad, velocidad o composición que, a lo largo del tiempo, se producen en el tráfico. Para poder acomodarse a esas variaciones se introdujeron modificaciones que mejoraban la adaptabilidad de los reguladores de tiempos fijos, llegando al diseño de los reguladores accionados por el tráfico, cuyo primer elemento indispensable es el detector.

Se puede intervenir en el funcionamiento de un regulador a tiempos fijos modificando de una manera u otra la duración de alguna de sus funciones. Así por ejemplo es relativamente fácil acelerar o retardar el ciclo o simplemente pararlo en el momento adecuado para prolongar una de las fases. Es más efectivo yuxtaponer en uno solo varios reguladores a tiempos fijos. Cada uno de ellos funcionará cuando sea seleccionado manualmente o por medio de un reloj. De esta última manera, lo que en realidad se tiene es una serie de programas fijos para varias situaciones de tráfico típicas y previsibles.

2.5.2 Cruces semiactuados

Los semáforos situados en intersecciones aisladas y cuyos cambios obedecen a las variaciones que se producen en el tráfico se llaman “accionados”. Pueden no estar accionados por todo el tráfico y entonces se les denomina “semiaccionados”, porque solamente tienen en cuenta una parte del tráfico, generalmente la menos importante. Este tipo de regulación suele instalarse cuando se trata de un cruce de una calle de relativa importancia con otra, que se considera menos importante. Los detectores se colocan en la vía secundaria y el regulador da paso libre a dicha vía siempre que, además de producirse una detección, se haya agotado el tiempo mínimo de verde asignado a la vía principal.

Un caso especial de funcionamiento semiaccionado es el de paso de peatones con pulsador, en el que el tráfico secundario está constituido por los peatones.

2.5.3 Cruces actuados

En los semáforos accionados, todo el tráfico que llega a la intersección es detectado y los tiempos se reparten de acuerdo con unos determinados criterios. Es posible, con este tipo de regulación, fijar entre ciertos límites algunas de las partes que componen el ciclo total. Una vez descrito con

detalle el funcionamiento de un regulador totalmente accionado por el tráfico, es muy fácil comprender como actúa un regulador semiaccionado, ya que este segundo no es más que una simplificación del primero.

Para poner a punto este tipo de reguladores es aconsejable hacerlo en horas punta, con lo que cuando baje el tráfico funcionarán perfectamente. Normalmente el cálculo del tiempo máximo de verde se hará como si se tratase de un regulador de tiempos fijos cuyo funcionamiento se previese para un tráfico en el que coincidiesen las horas punta de cada movimiento.

La experiencia indica que a falta de datos con los que poder hacer un buen estudio, se consigue un mejor funcionamiento fijando unos tiempos más bien bajos.

Cuanto más lejos estén los detectores (siempre dentro de ciertos límites) más “sensible” parecerá la regulación. Sin embargo como los vehículos almacenados entre el detector y la línea de detención tienen que poder atravesar el cruce durante el “tiempo mínimo”, cuanto más lejos esté el detector mayor tendrá que ser ese tiempo mínimo, con el consiguiente perjuicio para esa misma sensibilidad en la regulación.

El procedimiento descrito corresponde a reguladores accionados solamente por la intensidad del tráfico. Hay otros aparatos que, además, tienen en cuenta el número de vehículos que están parados, así como el tiempo que lo están; también consideran los intervalos entre unos vehículos y otros, es decir, consideran la densidad del tráfico.

Con una regulación por intensidad densidad conviene que los detectores, situados en todos los accesos, estén algo más alejados de la intersección que en el caso de regulación sólo por intensidad.

Los sistemas que tienen en cuenta la densidad están muy indicados cuando se trata de intersecciones muy sobrecargadas o cuando, por cualquier razón, el tráfico llega irregularmente y formando grupos más o menos compactos. En este último caso, la ventaja es clara, pues los automóviles agrupados representan una densidad alta, con lo que consiguen fácilmente más tiempo de paso o, por lo menos, una reducción importante en el tiempo de espera.

2.5.4 Comparativa entre los actuados y los no actuados

Siempre que sea aconsejable la instalación de semáforos, es preferible que su regulación, al menos en cierta medida, pueda depender del tráfico.

Un ejemplo claro en que debe escogerse el semáforo accionado es el de aquellos puntos en los que la instalación viene obligada exclusivamente por los peatones y éstos pasan a intervalos muy irregulares.

A pesar de todo conviene poner de manifiesto que los semáforos no accionados tienen algunas ventajas:

- Proporcionan una mejor coordinación con los de las intersecciones adyacentes, debido a la constancia de su ciclo y su reparto.
- El funcionamiento de los semáforos no se ve nunca afectado por anomalías en el detector, como pueden ser unas obras o un vehículo averiado.

- En general dan mejor resultado en lugares donde constantemente hay grandes masas de peatones.
- Por lo común son más económicos.

Como contrapartida de lo anterior se puede indicar una serie de ventajas a favor de los semáforos accionados. Entre ellas destacan las siguientes:

- Son más eficaces cuando existen fluctuaciones de tráfico de previsión imposible o difícil.
- Pueden ser mejores en intersecciones donde haya movimientos esporádicos o cuya intensidad varía mucho a lo largo del día.
- Su rendimiento es claramente mejor en los cruces de vías de muy poca importancia con otras que la tienen relativamente grande.
- En los lugares donde los semáforos de tiempos fijos deben ser puestos en destellos durante varias horas por falta de tráfico, los accionados son más adecuados.

2.6 Sincronización entre intersecciones

La regulación de tráfico tiene como primer objetivo la solución de un problema inequívocamente localizado, generalmente en un cruce de vías o intersección, sin tener que preocuparse por el problema que pueda existir aguas arriba o aguas abajo de la intersección o a su derecha o izquierda. Sin embargo, una vez solucionado el problema local y especialmente cuando otros cruces semaforizados están próximos, pueden darse interferencias no deseadas entre ellos que es preciso solucionar.

Está demostrado que cuando varias intersecciones semaforizadas están próximas entre sí, es conveniente sincronizar los reguladores con el fin de coordinar el encendido y apagado de los semáforos. La coordinación tiene por objeto el que un vehículo que circule por una arteria semaforizada en un sentido concreto y a una velocidad determinada, encuentre el menor número de semáforos en rojo y de que los tiempos de parada sean mínimos. Dicho en sentido positivo sería el que un vehículo que circule en una red semafórica sincronizada, encuentre en el sentido de la marcha el mayor número de semáforos en verde.

En la coordinación existen dos acciones distintas: la primera, que denominamos "sincronización", es el hecho físico de unir dos o más reguladores a través de un cable que transporta una señal eléctrica que llamamos "sincronismo" y que hace que los reguladores funcionen al unísono cada período de tiempo que llamamos ciclo; la segunda acción es propiamente la "coordinación" que forma parte del plan de ingeniería de tráfico y que determina cómo han de funcionar los semáforos para conseguir un efecto práctico.

La sincronización puede realizarse de dos formas diferentes:

- Por medio de los mismos reguladores.
- A través de un equipo diferente a los reguladores.

2.6.1 Sincronización mediante los mismos reguladores

En este caso, los equipos están interconectados a través de un cable denominado "cable de sincronismo" por el que se transmite la señal que sincroniza la red. Uno de los reguladores será el emisor y enviará la señal periódicamente al resto de reguladores que serán todos receptores. Antiguamente los receptores se paraban en una posición determinada del ciclo y no arrancaban hasta recibir la señal de sincronismo. Esta forma de proceder tenía serios inconvenientes por lo que en la actualidad los receptores no paran, sino que esperan un tiempo determinado para recibir la señal y si durante este tiempo no llega, siguen funcionando de forma autónoma, pero de forma más lenta o más rápida, es decir, autocorrigiéndose, hasta conseguir sincronizarse.

La señal de sincronismo puede ser diferente de un regulador a otro, especialmente teniendo en cuenta la diversidad de modelos y tecnologías que se encuentran en la calle. En la actualidad todavía se pueden encontrar señales de sincronismo a 220 V de CA para mandos electromecánicos y electrónicos; señales de 12 ó 24 V de CC y, posiblemente, señales mucho más complejas y sofisticadas para reguladores con microprocesador.

Existe la posibilidad de realizar la conexión entre los reguladores sin cable físico que los una. Aunque en el pasado se realizaron intentos de sincronizar los reguladores sin necesidad de estar unidos por medio de cable (hubo reguladores con motores síncronos que pretendían sincronizarse sin cable), ha sido con el desarrollo de la electrónica cuando más se ha extendido este procedimiento. De hecho se pueden considerar dos métodos distintos: el sincronismo vía radio, cuyas experiencias no dieron el resultado esperado, y el sincronismo por medio de reloj. Este último se puede subdividir a su vez en dos tipos: el de reloj interno basado en la frecuencia de la red o en un cristal de cuarzo (que en definitiva poco se distingue del de los motores síncronos) y el de reloj de tiempo real que si se sincroniza con una señal astronómica exterior puede ser bastante eficaz.

2.6.2 Sincronización mediante un equipo diferente a los reguladores

Este equipo puede ser un generador de sincronismo que es el equipo más elemental y cuya función es única y exclusivamente generar dicha señal, o una central de tráfico, si bien ésta cumple otras funciones más complejas. En este caso, los reguladores serán todos receptores ya que la señal de sincronismo será enviada por este último equipo.

3 DISEÑO DEL SISTEMA

3.1 Objetivo del trabajo

El objetivo que se persigue es proponer un diseño para un sistema de control de semáforos que permita automatizar de manera flexible, eficaz y económica los semáforos de intersecciones, de tal manera que se puedan programar en cualquier momento y cambiar todas las características de cada semáforo del cruce, tal como, duración de las fases, color en cada fase, duración en amarillo, etc.

Con la implantación de este sistema, el operario necesitará conectar el PC a una placa (pic18f47j53) mediante USB y con una aplicación de escritorio configurar como desee la intersección.

3.2 Planteamiento del problema

Como se ha visto, en la regulación de semáforos es necesario el uso de ciertos elementos que tienen la desventaja de ser poco flexibles en el caso de las intersecciones programadas a tiempos fijos. Los reguladores vistos en el apartado anterior eran de diferentes tipos, siendo el regulador por procesador el más ventajoso actualmente, si bien es necesaria una estandarización de este tipo de elemento.

La cuestión es que en caso de que sea necesario o se quieran cambiar los parámetros y características de la intersección será necesaria la sustitución de los reguladores antiguos, por otros programados a tal efecto, lo cual incrementa los costes de los cambios en los parámetros y hace que sea tedioso y poco flexible la configuración de los cruces.

Además de todo lo dicho, el tiempo necesario para realizar los cambios hace necesario abrir los armarios de chapa mencionados en los apartados anteriores para cambiar los reguladores, con la necesidad de poner a guardias de circulación mientras se realizan los cambios, es decir, mayores costes derivados del tiempo extra invertido.

Este hecho hace que el presente trabajo intente buscar una solución a dicho problema, para los cruces de semáforos de tiempos fijos en intersecciones aisladas, si bien podría ampliarse en el futuro para que este trabajo pudiera abarcar los cruces semiactuados y actuados.

Se intenta buscar una solución para hacer lo más flexible posible el sistema de reguladores de semáforos mencionados, y buscar una estandarización en el medida de los posible.

Sería ideal si se pudiera evitar tener que cambiar los reguladores y poder programarse in situ, o cambiar el sistema de reguladores por otro sistema más flexible.

3.3 Solución al problema

La solución que propone el presente trabajo no es más que cambiar el sistema de reguladores por un método flexible, rápido, eficaz y económico para realizar cambios en las intersecciones y en la programación de los semáforos de los cruces.

3.3.1 Utilización de microcontroladores PIC

La solución propuesta se basa en la utilización de microcontroladores PIC, que son una alternativa muy eficaz, económica y sobre todo flexible para todo tipo de aplicaciones. Así sustituiríamos a los reguladores clásicos por los microcontroladores.

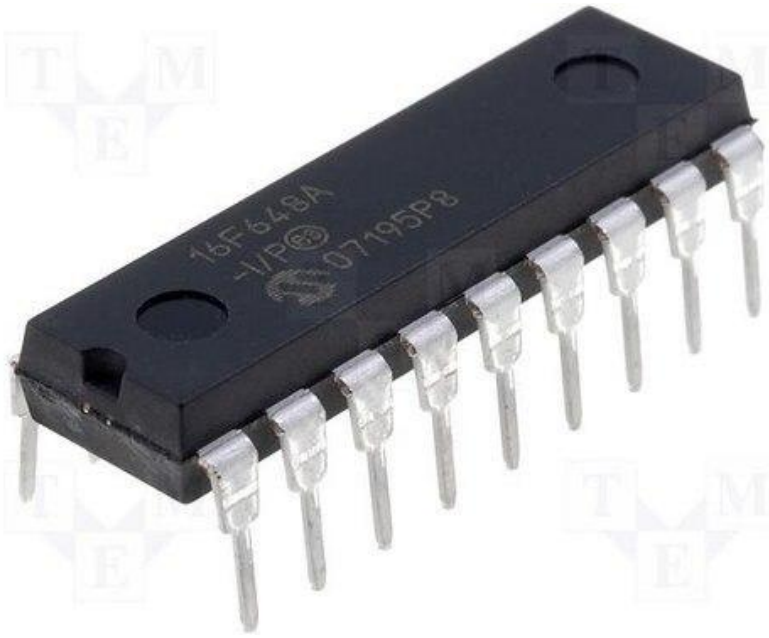


Figura 3.1 Ejemplo de PIC16

Dado que los PIC son de aplicación general se pueden programar según las necesidades de la intersección.

La idea es asignar a cada semáforo de la intersección un PIC16 y unirlos todos mediante el bus UART, siendo dirigidos todos por una unidad maestro (PIC18) que se encarga de dar las órdenes y configurar cada uno de los semáforos del cruce (cada PIC16).

3.3.2 Utilización del bus UART

Este bus será utilizado debido a la gran aceptación que tiene este protocolo y a la eficaz flexibilidad que presenta para comunicaciones entre distintos microcontroladores, entre sí y entre ellos y la unidad maestro (PIC18).

Más adelante se explicará con detalle cómo funciona este bus, pero el principal motivo es la sencillez que presenta para programarlo y enviar datos de manera relativamente rápida y segura, ya que es un bus diferencial.

3.3.3 Comunicación USB

El Universal Serial Bus (USB) es uno de los buses más extendidos actualmente y tiene aplicaciones desarrolladas en todos los ámbitos por lo tanto es muy recomendable usar este bus de comunicación entre la unidad maestro (PIC18) y el PC. A pesar de ser tan utilizado es un bus sumamente complejo ya que ofrece posibilidades casi ilimitadas, por no mencionar, que está en constante evolución.

El operario tan solo conectará su computadora, portátil, PDA…, incluso smartphone, con el armario donde esté guardado el sistema de la intersección vía USB (cable USB) y cargará la configuración que le haya introducido a la aplicación de escritorio (programa en Visual Studio 2010, se explicará con detalle más adelante), en la unidad maestro y esta a su vez le irá transmitiendo los datos al resto de PICs.

4 ESTRUCTURA DEL SISTEMA

4.1 Estructura global del sistema

Como se ha explicado la estructura del sistema que se plantea en el presente trabajo es la que se muestra a continuación:

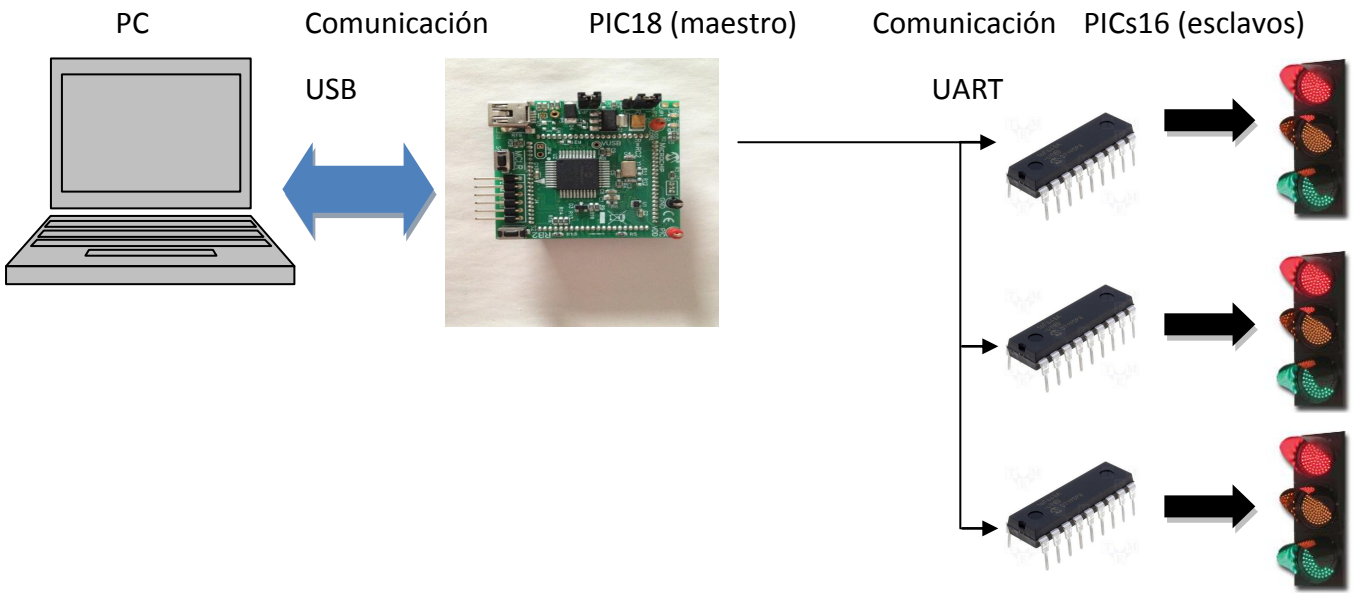


Figura 4.1 Esquema del sistema

Como se puede ver el PC mediante comunicación USB se conecta con la placa maestro (PIC18f47j53), transmitiendo los datos necesarios.

La placa maestro transmite los datos referentes al funcionamiento de los semáforos a los PIC16 en una fase que llamamos de "configuración", terminada esta el maestro irá mandando ciertas órdenes según proceda (se explicará detalladamente a continuación) a los PIC16 mediante el bus UART. Con la información recibida y la que van a ir recibiendo durante el funcionamiento, los PIC16 ya pueden mandar las señales digitales a las luces del semáforo según proceda, para ir conmutando entre los distintos colores según la fase en la que estén en ese momento y la duración determinada.

Se explicarán a continuación las diferentes partes de forma detallada.

4.2 PC

En esta parte lo que se va a hacer es un programa en Visual Studio 2010 (la programación está disponible en el anexo), que permite configurar la intersección de semáforos como se desee.

La estructura de este programa será la siguiente:

- En una primera parte se podrá elegir el número de semáforos que forman la intersección, así como cada una de las duraciones de las tres fases.

- En una segunda parte se podrá elegir para cada semáforo (previamente se ha indicado el número de semáforos constituyentes) el color que ha de tener en cada fase, el desfase que hay entre los cambios de rojo a verde y de verde a rojo y la duración del tiempo en amarillo.

Una vez introducidos los datos, el programa en Basic tiene un menú desplegable para elegir el puerto COM que se quiera (en este caso se debe elegir el puerto donde este el maestro conectado), luego mediante un botón que permite abrir el puerto seleccionado se establecerá la comunicación. Finalmente hay otro botón que permite el envío de información introducida en el programa, la cual se habrá almacenado en un vector, que es lo que se envía por el puerto USB hacia el maestro (PIC18).

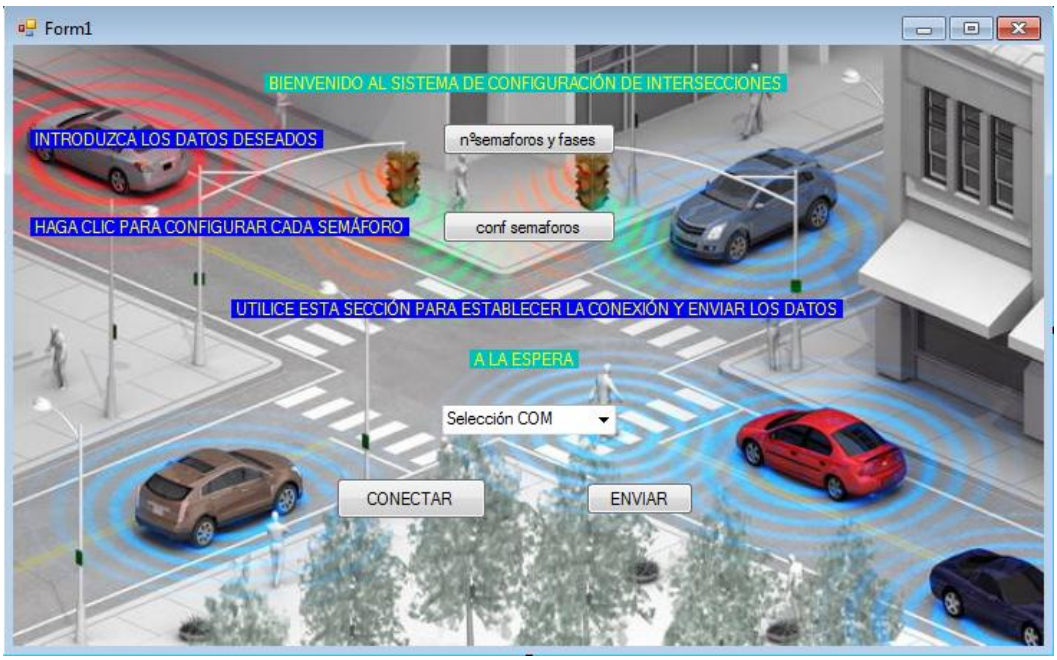


Figura 4.2 Aspecto de la ventana del programa en Basic

4.3 Placa PIC18F47J53 FS USB PIM DEMO BOARD

Esta parte es la encargada de hacer de interface entre el PC y los esclavos. El objetivo de este PIC18 es recibir el vector de datos desde el PC vía USB en la fase de configuración. Estará en un bucle hasta que reciba un byte de inicio de transmisión y entonces recibirá el primer paquete de datos (4 bytes) que corresponden con la duración de las fases y el número total de semáforos.

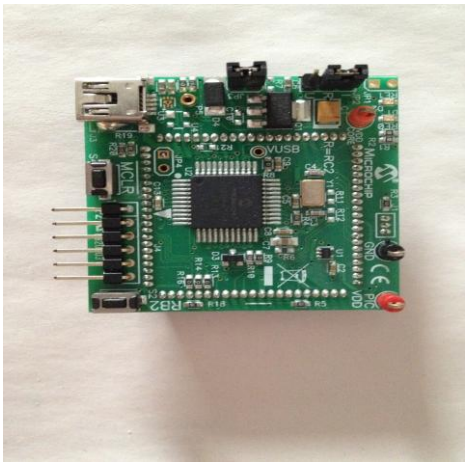


Figura 4.3 PIC18F47J53 DEMO BOARD

Seguidamente se van a ir enviando paquetes de 4 bytes (los bytes de configuración de cada semáforo) y una vez enviado un paquete se transmitirá por UART hacia el esclavo (PIC16) que le corresponda ya que primero se envía la cabecera de a quién corresponde los 4 bytes mencionados.

Los 4 bytes de cada semáforo son los siguientes:

1. Color por fase (0 si es verde 1 si es rojo y la posición indica a que fase corresponde el color).
2. Desfase de rojo a verde (este byte contiene el número de segundos de desfase).
3. Desfase de verde a rojo (este byte contiene el número de segundos de desfase).
4. Tiempo en amarillo (este byte indica los segundos que va a estar en amarillo).

No obstante el maestro enviará 2 bytes antes de estos para indicar:

1. Inicio de configuración del sistema (en binario 10000000, solo se envía al principio).
2. Dispositivo número x (en binario 100xxxxx, las x indican el dispositivo seleccionado).

Una vez terminada la configuración se enviarán:

1. Byte de fin de configuración del dispositivo (en binario 01100000).
2. Byte de fin de configuración del sistema (en binario 01000000).

Después de esta fase de configuración el microcontrolador pasa a un modo en el que envía un byte cada vez que ha pasado un segundo y si coincide que han pasado los segundos determinados para un cambio de fase también envía un byte indicando a que fase se debe cambiar.

Para crear el retardo de un segundo hay que tener en cuenta la frecuencia del oscilador, en este caso para un oscilador de 8 MHz debemos pasar de 1000ms a 8 MHz, según la siguiente igualdad:

$$\frac{1}{\frac{Fosc}{4}} = \frac{1}{\frac{8M}{4}} = 0.5\mu s \qquad (4.1)$$

$$\frac{1ms}{0.5\mu s} = 2000000 \quad (4.2)$$

Con el resultado obtenido aplicamos la instrucción de delay adecuada en lenguaje C y quedaría así: Delay10KTCYx(200), con la inclusión de la librería correspondiente para poder utilizar este tipo de instrucciones.

En este modo normal de funcionamiento mencionado los bytes mandados van dirigidos a todos los esclavos, es decir, todos los esclavos cogerán esta información del bus y la utilizarán.

Todos estos bytes mencionados se envían a través del bus UART a los PIC16 (esclavos).

4.4 Microcontroladores PIC16 de cada semáforo

Estos microcontroladores se encargarán de conmutar las luces del semáforo con los colores adecuados en cada fase. Para ello requieren de todos los datos transmitidos vía UART por el maestro (PIC18).

Cada vez que llegue información por el bus UART en la fase de configuración solo recibirá esta información el esclavo si la cabecera (identificador) del microcontrolador (esclavo) en cuestión, que utiliza un sistema de "switches" de 4 bits (en principio no son necesarios más) implementados en las placas donde van los PIC16, coincide con el identificador enviado por el maestro (PIC18) de tal forma que en caso de necesidad de cambiar alguna placa, el operario solo cambiará la placa y pondrá el código correcto en el switch, es decir, pulsará los conmutadores correctos según el identificador que tenga la placa (semáforo) en cuestión, sin necesidad de modificar la programación de los microcontroladores.

Después de que se ha verificado este identificador, entonces recibirá el paquete de datos mencionado por interrupción en la recepción del UART. Al recibir los 4 bytes de configuración, este esclavo (PIC16) quedará listo para el funcionamiento normal una vez terminada toda la configuración de los demás esclavos.

En modo normal irán recibiendo el byte que indica que ha pasado un segundo y el byte que indica el cambio a una determinada fase cuando proceda. Cuando reciben estos bytes se producirá una interrupción en los PIC16 para saltar del funcionamiento normal a dar la orden del cambio de color si es el caso y asegurando así que han recibido correctamente el dato. En este modo todos los esclavos cogen la información del bus sin necesitar identificador, ya que esta información es común y necesaria para todos los PIC16.

Para determinar de qué tipo de semáforo se trata, de peatones, normal de 3 colores o de giro con flecha, se alimentarán o se pondrán a masa unos pines de los microcontroladores, de tal forma que según están a 0 ó 1 dichos pines (se necesitarán solamente dos bits por tanto dos pines para determinar de entre los 3 tipos de semáforos), es decir, según la combinación de 2 bits que asociemos a cada tipo de semáforo el programa dentro del PIC cogerá esta información y ejecutará la parte del programa adecuada a el tipo de semáforo en cuestión.

Cada PIC16 ejecutará el programa adecuado enviando los niveles de alto o bajo a las salidas digitales habilitadas donde estarán conectadas las luces del semáforo, en el caso del semáforo de 3 colores tendremos 3 salidas digitales disponibles e iremos enviando nivel alto o bajo a la salida

que corresponda en cada fase, obteniendo así el color correcto en cada fase para todos los semáforos que compongan el sistema.

5 RECREACIÓN DE UNA INTERSECCIÓN PARA REPRESENTAR EL FUNCIONAMIENTO DEL SISTEMA

5.1 Diseño de las PCB's para los PIC16

El primer paso es la realización del diseño de las 9 PCB's (dado que la intersección es de 9 semáforos), donde van ensamblados los LED y los PIC16, con el programa Eagle. Con este software construimos el esquemático y a partir de él generamos la PCB.

Para realizar el esquemático se cogen los elementos que van a ir ensamblados en la PCB y se unen según corresponda, quedando como se puede ver a continuación:

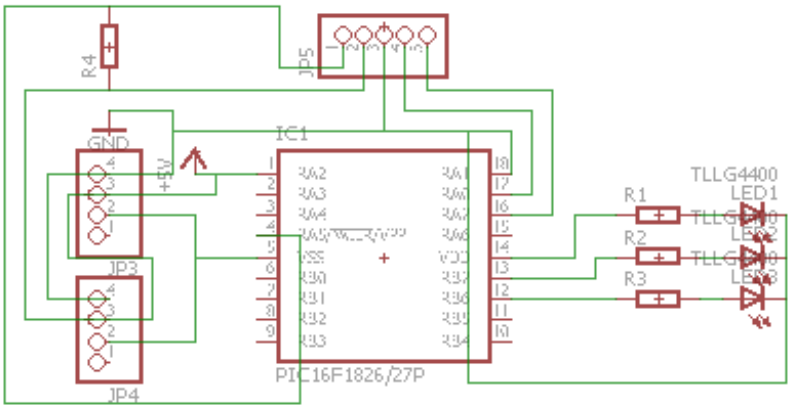


Figura 5.1 Esquemático en Eagle

Dado que no se dispone del modelo PIC16F1704 en la biblioteca de Eagle se ha seleccionado el siguiente modelo más parecido para respetar las dimensiones y las posiciones de los pines.

Una vez construido el esquemático se procede a generar la PCB de una cara, la cual, aparece descolocada para que se unan las pistas como mejor convenga. Una vez hecho esto se obtiene la PCB definitiva que se muestra a continuación:

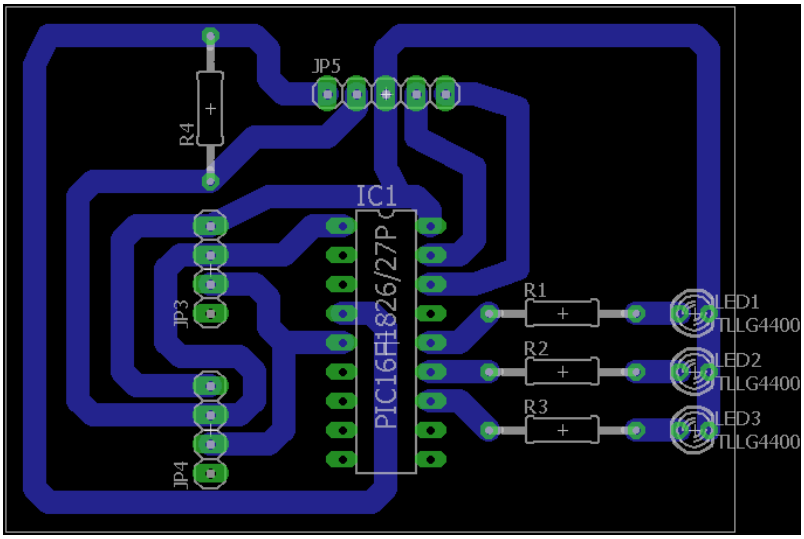


Figura 5.2 PCB generada en Eagle

5.2 Fabricación de las PCB's

El siguiente paso es el proceso de fabricación de los circuitos impresos, que en este caso se han fabricado por el proceso de revelado por reactivos.

Se insola la placa de baquelita de la cual se obtienen los circuitos, en la insoladora. Seguidamente se pasa por un baño en una sustancia de revelado, quedando expuesto el cobre de las partes innecesarias, mientras que las pistas quedan protegidas por la resina. Por último realiza otro baño en un producto ácido, que reacciona con el cobre, de tal forma, que desaparece la capa de cobre y queda la parte no conductora de la placa al descubierto, mientras que las pistas de cobre estaban protegidas por la resina. Limpiando la placa con alcohol eliminamos la resina que cubre las pistas y se finaliza el proceso.

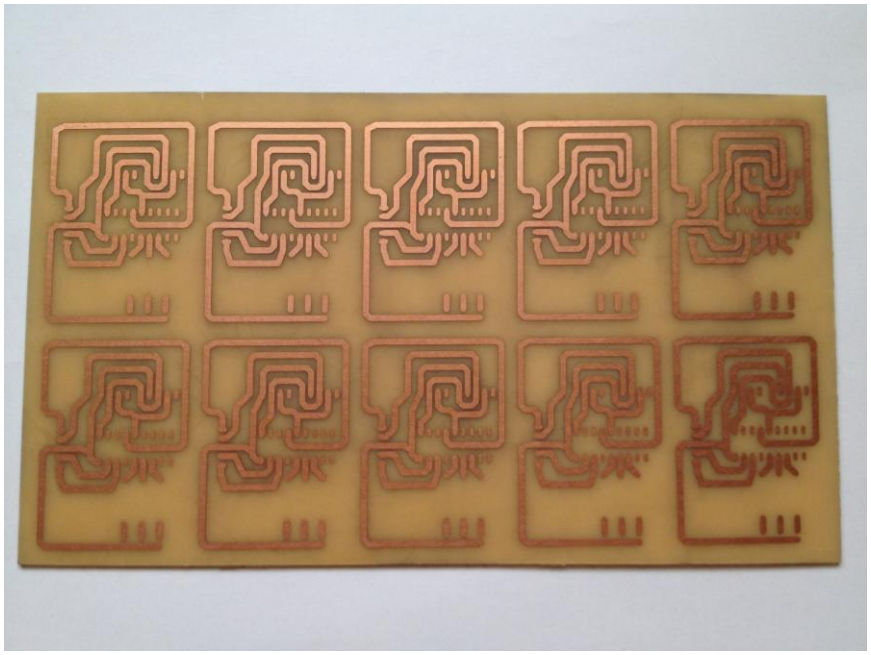


Figura 5.3 Circuitos impresos obtenidos por revelado por reactivos

Después de este proceso se ha cortado la baquelita para disponer de los circuitos de forma individual y se han realizado los taladros conforme al diseño en Eagle para poder ensamblar los componentes de cada circuito. Luego se ha realizado la soldadura de dichos componentes en la placa, quedando cada PCB como se muestra en la imagen:

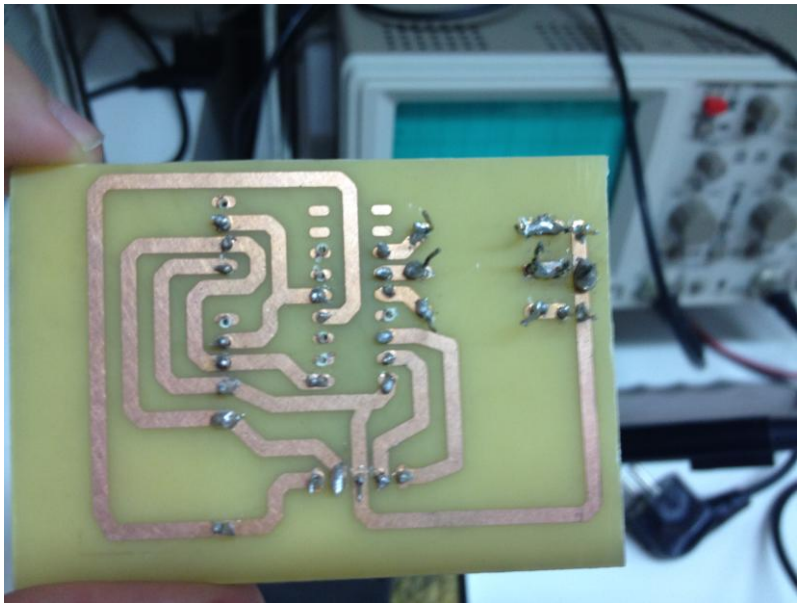


Figura 5.4 PCB realizados taladros y soldadura

5.3 Puesta en marcha

Se va a realizar la recreación de una intersección de 9 semáforos: 6 semáforos de 3 colores, 2 semáforos de peatones y 1 semáforo direccional de flecha. La intersección recreada pertenece a la ciudad de Badajoz más concretamente la que se muestra a continuación:



Figura 5.5 Intersección recreada

Se ha realizado un testeo del sistema en placas de prueba para realizar la fase de depuración de la programación de los PIC16 y del PIC18, una vez que se han eliminado los fallos en la programación posteriormente se ha realizado la programación de la memoria de los microcontroladores. Todo ello se ha hecho con la herramienta PICkit 3 de Microchip.

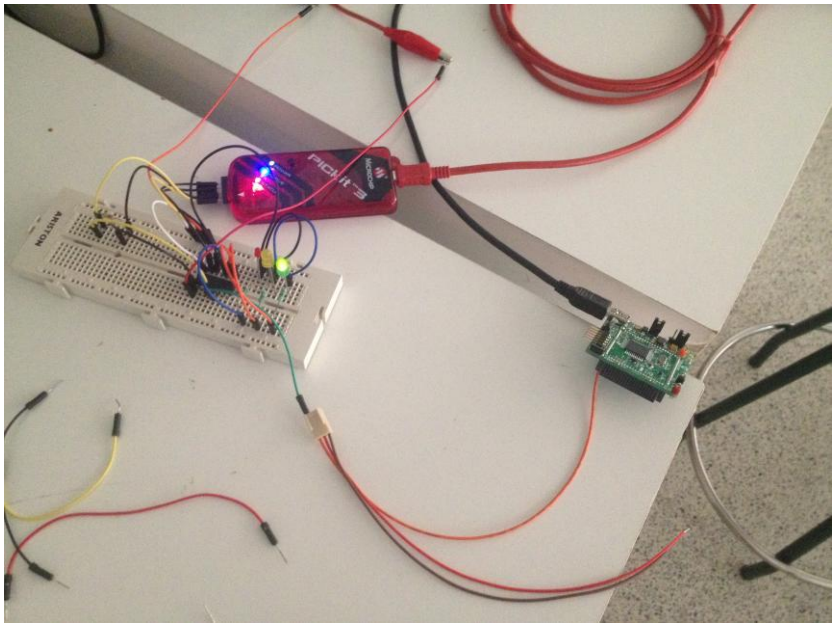


Figura 5.6 Testeo del sistema en placas de prueba

Una vez que se ha realizado el testeo y todo funciona correctamente se procede al lanzamiento del sistema en las PCB's fabricadas, para recrear la intersección mencionada, habiendo previamente configurado los 9 semáforos en la aplicación de escritorio.

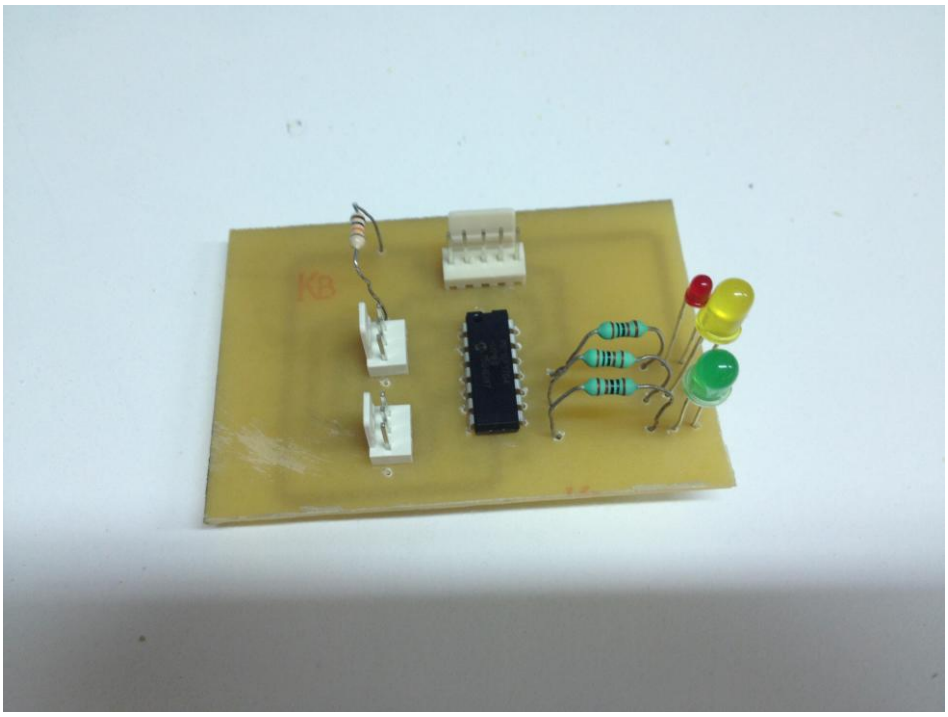


Figura 5.7 Cada una de las 9 PCB's fabricadas que recrean la intersección

6 MATERIALES Y HERRAMIENTAS UTILIZADAS

6.1 PIC18F47J53 FS USB PIM DEMO BOARD

Es una placa de demostración del fabricante Microchip, la cual se ha utilizado como unidad maestro en el sistema. El principal motivo de seleccionar esta placa es que cuenta con un canal USB implementado directamente, es decir, cuenta con un conector USB integrado de serie, lo que la hace ideal para la aplicación que se ha comentado.

El microcontrolador que gobierna la placa es un PIC18, un microcontrolador de gama alta. Entre sus características están la existencia de varios canales UART, I2C, memoria RAM asociada al puerto USB, varios conversores AD, etc.

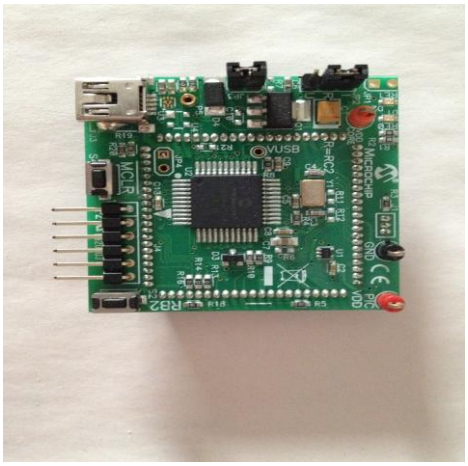


Figura 6.1 PIC18F47J53 FS USB PIM DEMO BOARD

Dada la complejidad para programar este tipo de microcontroladores se ha optado por utilizar el lenguaje C para programarlo. Para ello se ha utilizado el ensamblador C18 que provee Microchip para su gama de microcontroladores PIC18. Ha sido necesaria la utilización las librerías y archivos que proporciona el fabricante Microchip en sus librerías de soluciones “Microchip Solutions v2010-08-04” y “Microchip Solutions v2010-10-19”.

La utilización de estas soluciones que aporta Microchip ha sido crucial para programar ciertas partes, tal como la utilización módulo USB, el cual es sumamente complejo y en estas soluciones hay proyectos básicos incluidos que permiten desarrollar aplicaciones específicas a partir de un proyecto básico de transmisión de datos vía USB.

6.2 PIC16F1704-I/P

Se ha utilizado estos microcontroladores PIC16 de la gama media del fabricante Microchip, como los esclavos del sistema, para administrar cada uno de los semáforos de la intersección.

Se ha utilizado el microcontrolador más básico y económico que incluyese un canal UART, dado que es la única característica que importa en esta parte del sistema.

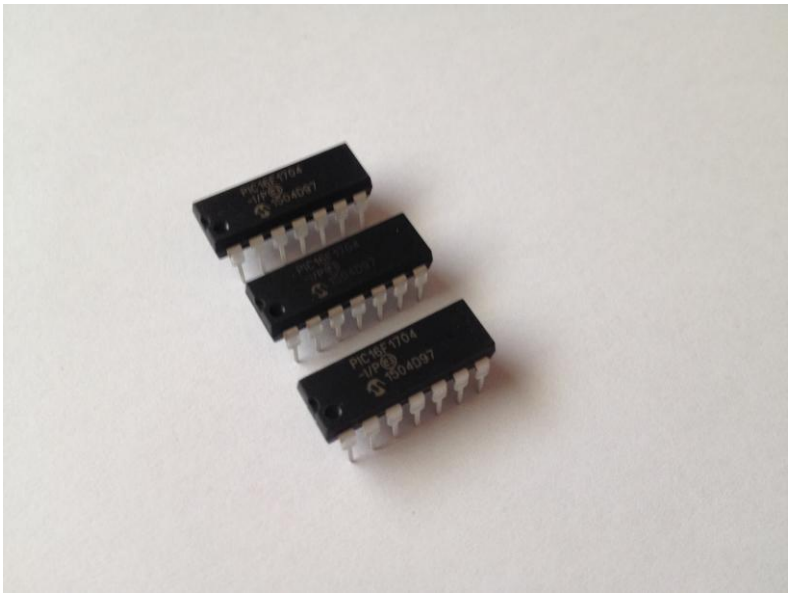


Figura 6.2 PIC16F1704-I/P

Solo debe recibir la información del maestro por el bus UART y realizar las conmutaciones según proceda de los colores del semáforo en el momento adecuado.

Estos microcontroladores se han programado en lenguaje ensamblador directamente dado que no conllevan demasiada complejidad. Lo importante es que cuenten con la posibilidad de generar interrupciones en la recepción de datos por UART.

Simplemente será necesario también contar con tres salidas digitales para poder enviar niveles alto y bajo a las diferentes luces del semáforo, y dos entradas digitales para determinar qué tipo de semáforo está asociado a el PIC en cuestión como se había comentado en el punto 4.

Estos microcontroladores irán ensamblados en unas PCB que cuentan con entrada de alimentación y con el canal UART necesario, para ir conectando todas las placas PCB unas con otras hasta completar el número de semáforos con que cuente la intersección.

6.3 Comunicación USB (Universal Serial Bus)

El Bus Universal en Serie (BUS) (en inglés: Universal Serial Bus), más conocido por la sigla USB, es un bus que establece un estándar industrial que define los cables, conectores y protocolos usados en un bus para conectar, comunicar y proveer de alimentación eléctrica entre computadoras, periféricos y dispositivos electrónicos.

Su desarrollo comenzó en un grupo de empresas del sector que buscaban unificar la forma de conectar periféricos a sus equipos, por aquella época poco compatibles entre sí, entre las que estaban Intel, Microsoft, IBM, Compaq, DEC, NEC y Nortel. La primera especificación completa, 1.0 se publicó en 1996, pero en 1998 con la especificación 1.1 comenzó a usarse de forma masiva.

El USB es utilizado como estándar de conexión de periféricos como: teclados, ratones, memorias USB, joysticks, escáneres, cámaras digitales, teléfonos móviles, reproductores multimedia, impresoras, dispositivos multifuncionales, sistemas de adquisición de datos, módems, tarjetas de red, tarjetas de sonido, tarjetas sintonizadoras de televisión y grabadoras de DVD externa, discos duros externos y disqueteras externas. Su éxito ha sido total, habiendo desplazado a conectores

como el puerto serie, puerto paralelo, puerto de juegos, Apple Desktop Bus o PS/2 a la consideración de dispositivos obsoletos, a eliminar de las modernas computadoras, pues muchos de ellos pueden sustituirse por dispositivos USB que implementen esos conectores.

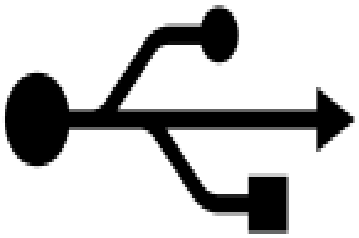


Figura 6.3 Símbolo del USB

Los dispositivos USB se clasifican en cuatro tipos según su velocidad de transferencia de datos:

- Baja velocidad (1.0): Tasa de transferencia de hasta 1,5 Mbit/s (188 kB/s). Utilizado en su mayor parte por dispositivos de interfaz humana (Human Interface Device, en inglés) como los teclados, los ratones (mouse), las cámaras web, etc.

- Velocidad completa (1.1): Tasa de transferencia de hasta 12 Mbit/s (1,5 MB/s) según este estándar. Ésta fue la más rápida antes de la especificación USB 2.0. Estos dispositivos dividen el ancho de banda de la conexión USB entre ellos, basados en un algoritmo de impedancias LIFO.

- Alta velocidad (2.0): Tasa de transferencia de hasta 480 Mbit/s (60 MB/s) pero con una tasa real práctica máxima de 280 Mbit/s (35 MB/s). El cable USB 2.0 dispone de cuatro líneas, un par para datos, y otro par de alimentación. Casi todos los dispositivos fabricados en la actualidad trabajan a esta velocidad

- Superalta velocidad (3.0): Tiene una tasa de transferencia de hasta 4,8 Gbit/s (600 MB/s). La velocidad del bus es diez veces más rápida que la del USB 2.0, debido a que han incluido 5 contactos adicionales, desechando el conector de fibra óptica propuesto inicialmente, y será compatible con los estándares anteriores. En octubre de 2009 la compañía taiwanesa ASUS lanzó la primera placa base que incluía puertos USB 3.0, tras ella muchas otras le han seguido y actualmente se ve cada vez más en placas base y portátiles nuevos, conviviendo junto con el USB 2.0.

Las señales del USB se transmiten en un cable de par trenzado con impedancia característica de 90 Ω ± 15%, cuyos hilos se denominan D+ y D-.8 Éstos, colectivamente, utilizan señalización diferencial en half dúplex excepto el USB 3.0 que utiliza un segundo par de hilos para realizar una comunicación en full dúplex. La razón por la cual se realiza la comunicación en modo diferencial es simple, reduce el efecto del ruido electromagnético en enlaces largos. D+ y D- suelen operar en conjunto y no son conexiones simples. Los niveles de transmisión de la señal varían de 0 a 0,3 V para bajos (ceros) y de 2,8 a 3,6 V para altos (unos) en las versiones 1.0 y 1.1, y en ±400 mV en alta velocidad (2.0). En las primeras versiones, los alambres de los cables no están conectados a tierra, pero en el modo de alta velocidad se tiene una terminación de 45 Ω a tierra o un diferencial de 90 Ω para acoplar la impedancia del cable. Este puerto sólo admite la conexión de dispositivos de bajo consumo, es decir, que tengan un consumo máximo de 100 mA por cada puerto; sin embargo, en caso de que estuviese conectado un dispositivo que permite 4 puertos por cada

salida USB (extensiones de máximo 4 puertos), entonces la energía del USB se asignará en unidades de 100 mA hasta un máximo de 500 mA por puerto. Con la primera fabricación de un PC con USB 3.0 en 2009, se tiene 1 A (un amperio) por puerto, lo cual da 5 W (cinco vatios) en lugar de 0,5 A (500 mA, 2,5W) como máximo.

6.4 Comunicación UART

UART, son las siglas en inglés de Universal Asynchronous Receiver-Transmitter, en español: Transmisor-Receptor Asíncrono Universal, es el dispositivo que controla los puertos y dispositivos serie.

El controlador del UART es el componente clave del subsistema de comunicaciones series de una computadora. El UART toma bytes de datos y transmite los bits individuales de forma secuencial. En el destino, un segundo UART reensambla los bits en bytes completos. La transmisión serie de la información digital (bits) a través de un cable único u otros medios es mucho más efectiva en cuanto a costo que la transmisión en paralelo a través de múltiples cables. Se utiliza un UART para convertir la información transmitida entre su forma secuencial y paralela en cada terminal de enlace. Cada UART contiene un registro de desplazamiento que es el método fundamental de conversión entre las forma secuencial y paralela.

El UART normalmente no genera directamente o recibe las señales externas entre los diferentes módulos del equipo. Usualmente se usan dispositivos de interfaz separados para convertir las señales de nivel lógico del UART hacia y desde los niveles de señalización externos.

Las señales externas pueden ser de variada índole. Ejemplos de estándares para señalización por voltaje son RS-232, RS-422 y RS-485 de la EIA.

6.5 Diodos LED

El LED (Light-Emitting Diode: Diodo Emisor de Luz), es un dispositivo semiconductor que emite luz incoherente de espectro reducido cuando se polariza de forma directa la unión PN en la cual circula por él una corriente eléctrica. Este fenómeno es una forma de electroluminiscencia, el LED es un tipo especial de diodo que trabaja como un diodo común, pero que al ser atravesado por la corriente eléctrica, emite luz. Este dispositivo semiconductor está comúnmente encapsulado en una cubierta de plástico de mayor resistencia que las de vidrio que usualmente se emplean en las lámparas incandescentes. Aunque el plástico puede estar coloreado, es sólo por razones estéticas, ya que ello no influye en el color de la luz emitida. Usualmente un LED es una fuente de luz compuesta con diferentes partes, razón por la cual el patrón de intensidad de la luz emitida puede ser bastante complejo.



Figura 6.4 Símbolo del diodo LED

Para obtener una buena intensidad luminosa debe escogerse bien la corriente que atraviesa el LED y evitar que este se pueda dañar; para ello, hay que tener en cuenta que el voltaje de operación va desde 1,8 hasta 3,8 voltios aproximadamente (lo que está relacionado con el material de fabricación y el color de la luz que emite) y la gama de intensidades que debe circular por él varía según su aplicación. Los Valores típicos de corriente directa de polarización de un LED están comprendidos entre los 10 y 20 miliamperios (mA) en los diodos de color rojo y de entre los 20 y 40 miliamperios (mA) para los otros LED. Los diodos LED tienen enormes ventajas sobre las lámparas indicadoras comunes, como su bajo consumo de energía, su mantenimiento casi nulo y con una vida aproximada de 100,000 horas. Para la protección del LED en caso haya picos inesperados que puedan dañarlo. Se coloca en paralelo y en sentido opuesto un diodo de silicio común

En general, los LED suelen tener mejor eficiencia cuanto menor es la corriente que circula por ellos, con lo cual, en su operación de forma optimizada, se suele buscar un compromiso entre la intensidad luminosa que producen (mayor cuanto más grande es la intensidad que circula por ellos) y la eficiencia (mayor cuanto menor es la intensidad que circula por ellos).

Para la recreación de la intersección que se va hacer para representar el funcionamiento del sistema explicado en este trabajo son necesarios poner en la maqueta diodos LED para emular las luces de los semáforos. La intersección que se va a emular consta de 6 semáforos de tres colores, 2 semáforos para peatones y un semáforo de flecha, lo que en total hace necesarios:

- 9 diodos LED verdes
- 9 diodos LED rojos
- 6 diodos LED amarillos



Figura 6.5 Diodo LED

6.6 Resistencias de 1kΩ y 10kΩ

Se van a utilizar resistencias disipadoras de 1kΩ para proteger los diodos LED que van colocados en las tres salidas digitales utilizadas en cada PIC16.



Figura 6.6 Resistencias de 1kΩ

Las resistencias de 10kΩ se utilizan como resistencias de Pull-up para poder programar en el propio circuito con el PICkit 3 cada uno de los PIC16. Habrá una resistencia de este valor en cada una de las placas donde están los PIC16.



Figura 6.7 Resistencia de 10kΩ

6.7 Baquelita positiva a una cara para fabricación de PCB's

La baquelita fue la primera sustancia plástica totalmente sintética, creada en 1907 y nombrada así en honor a su creador, el belga ganador del Premio Nobel en Química Leo Baekeland.

Se trata de un fenoplástico que hoy en día aún tiene aplicaciones interesantes. Este producto puede moldearse a medida que se forma y endurece al solidificarse. No conduce la electricidad, es resistente al agua y los solventes, pero fácilmente mecanizable. El alto grado de entrecruzamiento de la estructura molecular de la baquelita le confiere la propiedad de ser un plástico termoestable:

una vez que se enfría no puede volver a ablandarse. Esto lo diferencia de los polímeros termoplásticos, que pueden fundirse y moldearse varias veces, debido a que las cadenas pueden ser lineales o ramificadas pero no presentan entrecruzamiento, y por ello se clasifica como termofijo.

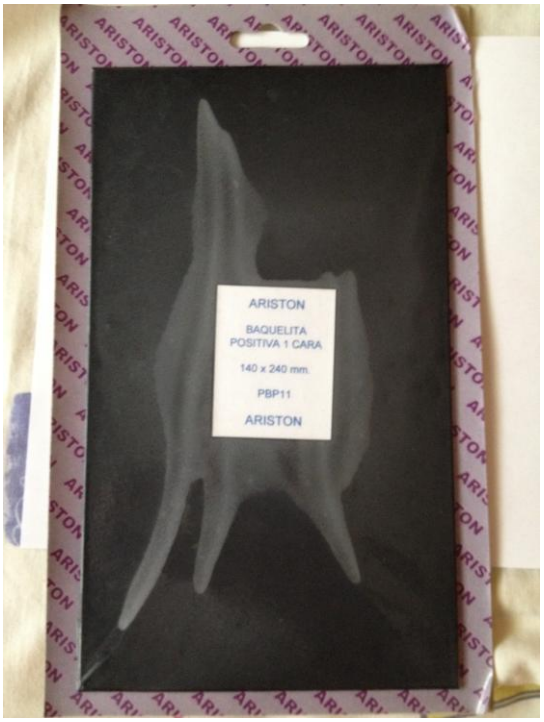


Figura 6.8 Baquelita positiva 1 cara

Se va utilizar la baquelita para realizar las PCB's donde van ensamblados cada uno de los PIC16, de tal forma que a partir de esta placa de baquelita se obtendrán 9 PCB's para cada uno de los 9 microcontroladores mencionados, así como sus respectivos diodos LED.

6.8 MPLAB IDE v8.92

MPLAB es un editor IDE gratuito, destinado a productos de la marca Microchip. Este editor es modular, permite seleccionar los distintos microcontroladores soportados, además de permitir la grabación de estos circuitos integrados directamente al programador.

Es un programa que corre bajo Windows, Mac OS y Linux. Presenta las clásicas barras de programa, de menú, de herramientas de estado, etc. El ambiente MPLAB posee editor de texto, compilador y simulación (no en tiempo real). Para comenzar un programa desde cero para luego grabarlo al µC en MPLAB v7.XX los pasos a seguir son:

1. Crear un nuevo archivo con extensión .ASM y nombre cualquiera
2. Crear un Proyecto nuevo eligiendo un nombre y ubicación
3. Agregar el archivo .ASM como un SOURCE FILE
4. Elegir el microcontrolador a utilizar desde SELECT DEVICE del menú CONFIGURE

Una vez realizado esto, se está en condiciones de empezar a escribir el programa respetando las directivas necesarias y la sintaxis para luego compilarlo y grabarlo en el PIC.

Se ha utilizado el MPLAB tanto para la programación del PIC18 (maestro), como de los PIC16 (esclavos). En el caso del maestro se ha recurrido a la programación en lenguaje C utilizando el ensamblador C18 proporcionado por Microchip. Los PIC16 se han programado en lenguaje ensamblador.

6.9 Microchip Solutions

Se trata de unos recursos que proporciona el fabricante Microchip, los cuales incluyen una carpeta con diferentes librerías y proyectos básicos de multitud de aplicaciones como pueden ser aplicaciones usando el canal USB, el canal UART, temporizadores, comunicaciones HID, etc. En este trabajo se ha utilizado entre otras el proyecto básico para la utilización de la comunicación USB que Microchip proporciona en este paquete, el cual se llama “CDC - Basic Demo”.

Además se han incluido diversos ficheros a parte que son necesarios para que funcione el programa y las instrucciones del maestro (PIC18). Se pueden ver en el Anexo de este trabajo.

6.10 Microsoft Visual Studio 2010

Visual Studio es un conjunto completo de herramientas de desarrollo para la generación de aplicaciones web ASP.NET, Servicios Web XML, aplicaciones de escritorio y aplicaciones móviles. Visual Basic, Visual C# y Visual C++ utilizan todos el mismo entorno de desarrollo integrado (IDE), que habilita el uso compartido de herramientas y hace más sencilla la creación de soluciones en varios lenguajes. Asimismo, dichos lenguajes utilizan las funciones de .NET Framework, las cuales ofrecen acceso a tecnologías clave para simplificar el desarrollo de aplicaciones web ASP y Servicios Web XML.



Figura 6.9 Logo del programa Visual Studio

Se ha utilizado el programa Visual Studio 2010 para hacer la aplicación de escritorio, que el operario debe ejecutar para configurar la intersección previamente de haber conectado vía USB el ordenador, Tablet, PDA, Smartphone… a la unidad maestro.

Se ha utilizado la parte de Visual Basic, en lenguaje Basic para hacer esta aplicación.

6.11 CadSoft Eagle

EAGLE, (siglas de Easily Applicable Graphical Layout Editor) es un programa de diseño de diagramas y PCBs con autoenrutador. Famoso en el mundo de los proyectos electrónicos DiY, debido a que muchas versiones de este programa tienen una licencia Freeware y gran cantidad de bibliotecas de componentes alrededor de la red.



Figura 6.10 Logo del programa Eagle

EAGLE contiene un editor de diagramas electrónicos. Los componentes pueden ser colocados en el diagrama con un solo click y fácilmente enrutables con otros componentes a base de "cables" o etiquetas.

EAGLE contiene un editor de PCBs con un auteoenrutador bastante eficiente. El editor es capaz de producir archivos GERBER entre otros, que son utilizados en el momento de la producción.

Eagle trae incluidas bibliotecas de componentes, sencillas de hacer y disponibles por parte de empresas, tales como SparkFun, o aficionados que las distribuyen alrededor de la red de forma gratuita.

Se ha utilizado el programa Eagle para el diseño de las PCBs donde van a ir insertados los microcontroladores PIC16 de cada semáforo.

En las placas se deberán poner las entradas de alimentación, masa, comunicación UART y salidas digitales, además de los 2 pines identificadores de tipo de semáforos mencionado en puntos anteriores.

6.12 PICkit 3

El PIckit 3 Original es un sistema de programación y depuración desarrollado por Microchip el cual usa lógica de in-circuit debugging incorporada dentro de cada una de las memorias flash con el objetivo de proveer un sistema de programación de alta calidad y bajo costo. El pickit 3 permite programar la mayoría de familias de los microcontroladores PIC y los DPSPIC, además de memorias EEPROM de Microchip

Características:

- USB (Full speed 12 Mbits/s interface to host PC)
- Ejecución en tiempo real
- MPLAB IDE compatible (free copy included)
- Built-in over-voltage/short circuit monitor

- Firmware upgradeable from PC/web download
- Soporta vajo voltaje de 2.0v a 6.0v
- LEDs indicadores (power, busy, error)
- Lectura y escritura de la memoria de los microcontroladores PIC
- Borrado del espacio de memoria con verificación
- Congelamiento de prifecicos en Break poin



Figura 6.11 PICkit 3

Se ha utilizado el PICkit 3 en la fase de depuración en los programas desarrollados en el PIC18 y en los PIC16 y también para la programación propiamente dicha de los microcontroladores mencionados.

7 CONCLUSIONES Y FUTURAS AMPLIACIONES

Con este trabajo se consigue un sistema de control de los semáforos en intersecciones aisladas y de tiempos fijos que es flexible, eficaz, económico, etc.

Pero no obstante, este trabajo puede ser ampliado en un futuro, añadiendo control del tráfico en las intersecciones aisladas, mediante detectores y sensores. Así se tendría en cuenta la variación del tráfico, llevando los datos proporcionados por los sensores a las entradas analógicas y digitales de los microcontroladores que se han utilizado, ya que tienen capacidad suficiente para añadir estas características. Se pueden regular así los semáforos según los valores recibidos por estos sensores y una vez procesados por los microcontroladores conmutar de forma inteligente los colores de las luces de la forma más eficiente según los datos adquiridos. También se pueden tener en cuenta otros datos como la hora del día, las condiciones climatológicas, etc.

Otra de las ampliaciones posibles es la introducción de sistemas de comunicación entre intersecciones, teniendo en cuenta los flujos de tráfico entre las distintas intersecciones. Para esto sería necesario implantar un sistema de comunicaciones entre los distintos maestros (PIC18) mediante comunicación inalámbrica o por cable poniendo elementos a modo de compuerta entre una intersección y otra de tal forma que la información o situación actual de tráfico en una intersección llegue como dato a la siguiente. No solo información de los flujos de tráfico sino también las condiciones climatológicas, la hora actual para descongestionar en horas punta, etc.

Se consigue así un tráfico más fluido encontrando en una misma dirección la mayor cantidad de semáforos en verde.

De esta forma el trabajo sería más complejo pero queda constancia de que es posible realizar una ampliación y es factible hacerlo.

8 BIBLIOGRAFÍA

[1] http://es.wikipedia.org/wiki/Semáforo

[2] http://noticias.coches.com/noticias-motor/historia-del-semaforo-cien-anos-dirigiendo-el-trafico/138897

[3] http://www.dgt.es/Galerias/la-dgt/empleo-publico/oposiciones/doc/2013/TEMA_24_GESTION_TECNICA_TRAFICO.doc

[4] http://www.dgt.es/es/la-dgt/empleo-publico/oposiciones/2014/20141216-temario-promocion-interna-2014-parte-3-gestion-tecnica-del-trafico.shtml

[5] http://www.dgt.es/Galerias/la-dgt/empleo-publico/oposiciones/doc/2014/TEMA-3.15.doc

[6] www.nortrafic.com

[7] todoproductividad.blogspot.com

[8] http://es.wikipedia.org/wiki/Universal_Serial_Bus

[9] http://es.wikipedia.org/wiki/Universal_Asynchronous_Receiver-Transmitter

[10] http://www.monografias.com/trabajos60/diodo-led/diodo-led.shtml

[11] http://es.wikipedia.org/wiki/MPLAB

[12] https://msdn.microsoft.com/es-es/library/fx6bk1f4%28v=vs.90%29.aspx

[13] http://es.wikipedia.org/wiki/EAGLE

[14] http://www.dynamoelectronics.com/index.php?product_id=267&page=shop.product_details&category_id=81&flypage=dynamo.tpl&option=com_virtuemart&Itemid=58

[15] https://es.wikipedia.org/wiki/Baquelita

9 ANEXO

9.1 Programación PIC18

/**

FileName: main.c

Dependencies: See INCLUDES section

Processor: PIC18, PIC24, and PIC32 USB Microcontrollers

Hardware: This demo is natively intended to be used on Microchip USB demo boards supported by the MCHPFSUSB stack. See release notes for support matrix. This demo can be modified for use on other hardware platforms.

Complier: Microchip C18 (for PIC18), C30 (for PIC24), C32 (for PIC32)

Company: Microchip Technology, Inc.

Software License Agreement:

The software supplied herewith by Microchip Technology Incorporated (the “Company”) for its PIC® Microcontroller is intended and supplied to you, the Company’s customer, for use solely and exclusively on Microchip PIC Microcontroller products. The software is owned by the Company and/or its supplier, and is protected under applicable copyright laws. All rights are reserved. Any use in violation of the foregoing restrictions may subject the user to criminal sanctions under applicable laws, as well as to civil liability for the breach of the terms and conditions of this license.

THIS SOFTWARE IS PROVIDED IN AN “AS IS” CONDITION. NO WARRANTIES,

```
 WHETHER EXPRESS, IMPLIED OR STATUTORY, INCLUDING, BUT NOT LIMITED
 TO, IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
 PARTICULAR PURPOSE APPLY TO THIS SOFTWARE. THE COMPANY SHALL NOT,
 IN ANY CIRCUMSTANCES, BE LIABLE FOR SPECIAL, INCIDENTAL OR
 CONSEQUENTIAL DAMAGES, FOR ANY REASON WHATSOEVER.

********************************************************************
 File Description:

 Change History:
  Rev   Description
  ----  -----------------------------------------
  1.0   Initial release

  2.1   Updated for simplicity and to use common
        coding style

  2.6a  Added button debouncing using Start-of-Frame packets

  2.7   Updated demo to place the PIC24F devices into sleep when the
        USB is in suspend.
********************************************************************/

/** INCLUDES *******************************************************/
#include "./USB/usb.h"
#include "./USB/usb_function_cdc.h"

#include "HardwareProfile.h"
```

```
/** CONFIGURATION *********************************************/

#if defined(PICDEM_FS_USB)      // Configuration bits for PICDEM FS USB Demo Board (based on PIC18F4550)

    #pragma config PLLDIV   = 2             //2 = 8 MHz        // (5 = 20 MHz crystal on PICDEM FS USB board)

            #pragma config CPUDIV   = OSC1_PLL2

    #pragma config USBDIV   = 2       // Clock source from 96MHz PLL/2

    #pragma config FOSC     = HSPLL_HS

    #pragma config FCMEN    = OFF

    #pragma config IESO     = OFF

    #pragma config PWRT     = OFF

    #pragma config BOR      = ON

    #pragma config BORV     = 3

    #pragma config VREGEN   = ON      //USB Voltage Regulator

    #pragma config WDT      = OFF

    #pragma config WDTPS    = 32768

    #pragma config MCLRE    = ON

    #pragma config LPT1OSC  = OFF

    #pragma config PBADEN   = OFF

//    #pragma config CCP2MX   = ON

    #pragma config STVREN   = ON

    #pragma config LVP      = OFF

//    #pragma config ICPRT    = OFF      // Dedicated In-Circuit Debug/Programming

    #pragma config XINST    = OFF      // Extended Instruction Set

    #pragma config CP0      = OFF

    #pragma config CP1      = OFF

//    #pragma config CP2      = OFF

//    #pragma config CP3      = OFF
```

```
    #pragma config CPB    = OFF
//  #pragma config CPD    = OFF
    #pragma config WRT0   = OFF
    #pragma config WRT1   = OFF
//  #pragma config WRT2   = OFF
//  #pragma config WRT3   = OFF
    #pragma config WRTB   = OFF     // Boot Block Write Protection
    #pragma config WRTC   = OFF
//  #pragma config WRTD   = OFF
    #pragma config EBTR0  = OFF
    #pragma config EBTR1  = OFF
//  #pragma config EBTR2  = OFF
//  #pragma config EBTR3  = OFF
    #pragma config EBTRB  = OFF



#elif defined(PIC18F87J50_PIM)                  // Configuration bits for PIC18F87J50 FS
USB Plug-In Module board
    #pragma config XINST  = OFF  // Extended instruction set
    #pragma config STVREN = ON          // Stack overflow reset
    #pragma config PLLDIV = 3       // (12 MHz crystal used on this board)
    #pragma config WDTEN  = OFF         // Watch Dog Timer (WDT)
    #pragma config CP0    = OFF         // Code protect
    #pragma config CPUDIV = OSC1    // OSC1 = divide by 1 mode
    #pragma config IESO   = OFF         // Internal External (clock) Switchover
    #pragma config FCMEN  = OFF         // Fail Safe Clock Monitor
    #pragma config FOSC   = HSPLL   // Firmware must also set OSCTUNE<PLLEN> to start PLL!
    #pragma config WDTPS  = 32768
//  #pragma config WAIT   = OFF         // Commented choices are
```

```
//     #pragma config BW      = 16              // only available on the
//     #pragma config MODE    = MM              // 80 pin devices in the
//     #pragma config EASHFT  = OFF             // family.
       #pragma config MSSPMSK  = MSK5
//     #pragma config PMPMX   = DEFAULT
//     #pragma config ECCPMX  = DEFAULT
       #pragma config CCP2MX  = DEFAULT


#elif defined(PIC18F46J50_PIM) || defined(PIC18F_STARTER_KIT_1) || defined(PIC18F47J53_PIM)
          #pragma config WDTEN = OFF        //WDT disabled (enabled by SWDTEN bit)
          #pragma config CFGPLLEN = ON              //habilitamos PLL
    #pragma config PLLDIV = 3         //2 = 8MHz Divide by 3 (3 = 12 MHz oscillator input)
    #pragma config STVREN = ON          //stack overflow/underflow reset enabled
    #pragma config XINST = OFF        //Extended instruction set disabled
    //#pragma config CPUDIV = OSC1       //No CPU system clock divide
          #pragma config CPUDIV = OSC4_PLL6         //necesario para tener un oscilador de 8 MHz

    #pragma config CP0 = OFF          //Program memory is not code-protected
    #pragma config OSC = HSPLL //INTOSCPLL         //HS oscillator, PLL enabled, HSPLL used by USB
                Se utiliza oscilador interno
    //#pragma config OSC = INTOSC
          //#pragma config T1DIG = ON         //Sec Osc clock source may be selected
    //#pragma config LPT1OSC = OFF       //high power Timer1 mode
    #pragma config FCMEN = OFF        //Fail-Safe Clock Monitor disabled
    #pragma config IESO = OFF         //Two-Speed Start-up disabled
    #pragma config WDTPS = 32768       //1:32768
    #pragma config DSWDTOSC = INTOSCREF //DSWDT uses INTOSC/INTRC as clock
    #pragma config RTCOSC = T1OSCREF    //RTCC uses T1OSC/T1CKI as clock
    #pragma config DSBOREN = OFF       //Zero-Power BOR disabled in Deep Sleep
```

```
    #pragma config DSWDTEN = OFF       //Disabled
    #pragma config DSWDTPS = 8192      //1:8,192 (8.5 seconds)
    #pragma config IOL1WAY = OFF       //IOLOCK bit can be set and cleared
    #pragma config MSSP7B_EN = MSK7    //7 Bit address masking
    #pragma config WPFP = PAGE_1       //Write Protect Program Flash Page 0
    #pragma config WPEND = PAGE_0      //Start protection at page 0
    #pragma config WPCFG = OFF         //Write/Erase last page protect Disabled
    #pragma config WPDIS = OFF         //WPFP[5:0], WPEND, and WPCFG bits ignored
#elif defined(LOW_PIN_COUNT_USB_DEVELOPMENT_KIT)
      //14K50
      #pragma config CPUDIV = NOCLKDIV
      #pragma config USBDIV = OFF
      #pragma config FOSC   = HS
      #pragma config PLLEN  = ON
      #pragma config FCMEN  = OFF
      #pragma config IESO   = OFF
      #pragma config PWRTEN = OFF
      #pragma config BOREN  = OFF
      #pragma config BORV   = 30
//      #pragma config VREGEN = ON
      #pragma config WDTEN  = OFF
      #pragma config WDTPS  = 32768
      #pragma config MCLRE  = OFF
      #pragma config HFOFST = OFF
      #pragma config STVREN = ON
      #pragma config LVP    = OFF
      #pragma config XINST  = OFF
      #pragma config BBSIZ  = OFF
```

```
    #pragma config CP0    = OFF
    #pragma config CP1    = OFF
    #pragma config CPB    = OFF
    #pragma config WRT0   = OFF
    #pragma config WRT1   = OFF
    #pragma config WRTB   = OFF
    #pragma config WRTC   = OFF
    #pragma config EBTR0  = OFF
    #pragma config EBTR1  = OFF
    #pragma config EBTRB  = OFF

#elif defined(EXPLORER_16)
  #if defined(__PIC24FJ256GB110__) || defined(__PIC24FJ256GB210__)
    _CONFIG1( JTAGEN_OFF & GCP_OFF & GWRP_OFF & FWDTEN_OFF & ICS_PGx2)
    _CONFIG2( 0xF7FF & IESO_OFF & FCKSM_CSDCMD & OSCIOFNC_ON & POSCMOD_HS & FNOSC_PRIPLL & PLLDIV_DIV2 & IOL1WAY_ON)
  #elif defined(__PIC24FJ64GB004__)
    _CONFIG1(WDTPS_PS1 & FWPSA_PR32 & WINDIS_OFF & FWDTEN_OFF & ICS_PGx1 & GWRP_OFF & GCP_OFF & JTAGEN_OFF)
    _CONFIG2(POSCMOD_HS & I2C1SEL_PRI & IOL1WAY_OFF & OSCIOFNC_ON & FCKSM_CSDCMD & FNOSC_PRIPLL & PLL96MHZ_ON & PLLDIV_DIV2 & IESO_ON)
    _CONFIG3(WPFP_WPFP0 & SOSCSEL_SOSC & WUTSEL_LEG & WPDIS_WPDIS & WPCFG_WPCFGDIS & WPEND_WPENDMEM)
    _CONFIG4(DSWDTPS_DSWDTPS3 & DSWDTOSC_LPRC & RTCOSC_SOSC & DSBOREN_OFF & DSWDTEN_OFF)
  #elif defined(__32MX460F512L__) || defined(__32MX795F512L__)
    #pragma config UPLLEN   = ON        // USB PLL Enabled
    #pragma config FPLLMUL  = MUL_15        // PLL Multiplier
    #pragma config UPLLIDIV = DIV_2         // USB PLL Input Divider
    #pragma config FPLLIDIV = DIV_2         // PLL Input Divider
```

```
    #pragma config FPLLODIV = DIV_1        // PLL Output Divider
    #pragma config FPBDIV   = DIV_1        // Peripheral Clock divisor
    #pragma config FWDTEN   = OFF          // Watchdog Timer
    #pragma config WDTPS    = PS1          // Watchdog Timer Postscale
    #pragma config FCKSM    = CSDCMD       // Clock Switching & Fail Safe Clock Monitor
    #pragma config OSCIOFNC = OFF          // CLKO Enable
    #pragma config POSCMOD  = HS           // Primary Oscillator
    #pragma config IESO     = OFF          // Internal/External Switch-over
    #pragma config FSOSCEN  = OFF          // Secondary Oscillator Enable (KLO was off)
    #pragma config FNOSC    = PRIPLL       // Oscillator Selection
    #pragma config CP       = OFF          // Code Protect
    #pragma config BWP      = OFF          // Boot Flash Write Protect
    #pragma config PWP      = OFF          // Program Flash Write Protect
    #pragma config ICESEL   = ICS_PGx2     // ICE/ICD Comm Channel Select
    #pragma config DEBUG    = ON           // Background Debugger Enable
  #else
    #error No hardware board defined, see "HardwareProfile.h" and __FILE__
  #endif
#elif defined(PIC24F_STARTER_KIT)
  _CONFIG1( JTAGEN_OFF & GCP_OFF & GWRP_OFF & COE_OFF & FWDTEN_OFF & ICS_PGx2)
  _CONFIG2( 0xF7FF & IESO_OFF & FCKSM_CSDCMD & OSCIOFNC_ON & POSCMOD_HS &
FNOSC_PRIPLL & PLLDIV_DIV3 & IOL1WAY_ON)
#elif defined(PIC24FJ256DA210_DEV_BOARD)
  _CONFIG1(FWDTEN_OFF & ICS_PGx2 & GWRP_OFF & GCP_OFF & JTAGEN_OFF)
  _CONFIG2(POSCMOD_HS & IOL1WAY_ON & OSCIOFNC_ON & FCKSM_CSDCMD &
FNOSC_PRIPLL & PLL96MHZ_ON & PLLDIV_DIV2 & IESO_OFF)
#elif defined(PIC32_USB_STARTER_KIT)
  #pragma config UPLLEN   = ON       // USB PLL Enabled
  #pragma config FPLLMUL  = MUL_15       // PLL Multiplier
```

```
    #pragma config UPLLIDIV = DIV_2        // USB PLL Input Divider
    #pragma config FPLLIDIV = DIV_2        // PLL Input Divider
    #pragma config FPLLODIV = DIV_1        // PLL Output Divider
    #pragma config FPBDIV   = DIV_1        // Peripheral Clock divisor
    #pragma config FWDTEN   = OFF          // Watchdog Timer
    #pragma config WDTPS    = PS1          // Watchdog Timer Postscale
    #pragma config FCKSM    = CSDCMD       // Clock Switching & Fail Safe Clock Monitor
    #pragma config OSCIOFNC = OFF          // CLKO Enable
    #pragma config POSCMOD  = HS           // Primary Oscillator
    #pragma config IESO     = OFF          // Internal/External Switch-over
    #pragma config FSOSCEN  = OFF          // Secondary Oscillator Enable (KLO was off)
    #pragma config FNOSC    = PRIPLL       // Oscillator Selection
    #pragma config CP       = OFF          // Code Protect
    #pragma config BWP      = OFF          // Boot Flash Write Protect
    #pragma config PWP      = OFF          // Program Flash Write Protect
    #pragma config ICESEL   = ICS_PGx2     // ICE/ICD Comm Channel Select
    #pragma config DEBUG    = ON           // Background Debugger Enable
#else
    #error No hardware board defined, see "HardwareProfile.h" and __FILE__
#endif

/** I N C L U D E S **********************************************************/

#include "GenericTypeDefs.h"
#include "Compiler.h"
#include "usb_config.h"
#include "USB\usb_device.h"
#include "USB\usb.h"
```

```
#include "HardwareProfile.h"
#include <p18f47j53.h>
#include <usart.h>
#include <delays.h>

/** V A R I A B L E S ********************************************************/
#pragma udata
char USB_In_Buffer[64];
char USB_Out_Buffer[64];

BOOL stringPrinted;
volatile BOOL buttonPressed;
volatile BYTE buttonCount;

//****Variables del proyecto
//char USB_Out_Buffer[64];
//char buffer[64];
char vectorenvio[64];
char semaforos, tfase1, tfase2, tfase3;
char i, inter, contfase;
unsigned char UART1Config = 0, baud = 0, disp, finconf;     //la variable "disp" indica el dispositivo al que van dirigidos los datos en cada iteracion


/** P R I V A T E  P R O T O T Y P E S ***************************************/
static void InitializeSystem(void);
void ProcessIO(void);
void USBDeviceTasks(void);
```

```
void YourHighPriorityISRCode();
void YourLowPriorityISRCode();
void BlinkUSBStatus(void);
void UserInit(void);

//Prototipos del proyecto
void confirma(void);
void esperaenvio(void);

/** VECTOR REMAPPING ***********************************************/
#if defined(__18CXX)
	//On PIC18 devices, addresses 0x00, 0x08, and 0x18 are used for
	//the reset, high priority interrupt, and low priority interrupt
	//vectors.  However, the current Microchip USB bootloader
	//examples are intended to occupy addresses 0x00-0x7FF or
	//0x00-0xFFF depending on which bootloader is used.  Therefore,
	//the bootloader code remaps these vectors to new locations
	//as indicated below.  This remapping is only necessary if you
	//wish to program the hex file generated from this project with
	//the USB bootloader.  If no bootloader is used, edit the
	//usb_config.h file and comment out the following defines:
	//#define PROGRAMMABLE_WITH_USB_HID_BOOTLOADER
	//#define PROGRAMMABLE_WITH_USB_LEGACY_CUSTOM_CLASS_BOOTLOADER

	#if defined(PROGRAMMABLE_WITH_USB_HID_BOOTLOADER)
		#define REMAPPED_RESET_VECTOR_ADDRESS			0x1000
		#define REMAPPED_HIGH_INTERRUPT_VECTOR_ADDRESS		0x1008
		#define REMAPPED_LOW_INTERRUPT_VECTOR_ADDRESS	0x1018
```

```
#elif defined(PROGRAMMABLE_WITH_USB_MCHPUSB_BOOTLOADER)
        #define REMAPPED_RESET_VECTOR_ADDRESS            0x800
        #define REMAPPED_HIGH_INTERRUPT_VECTOR_ADDRESS   0x808
        #define REMAPPED_LOW_INTERRUPT_VECTOR_ADDRESS 0x818
#else
        #define REMAPPED_RESET_VECTOR_ADDRESS            0x00
        #define REMAPPED_HIGH_INTERRUPT_VECTOR_ADDRESS   0x08
        #define REMAPPED_LOW_INTERRUPT_VECTOR_ADDRESS 0x18
#endif

#if defined(PROGRAMMABLE_WITH_USB_HID_BOOTLOADER)||defined(PROGRAMMABLE_WITH_USB_MCHPUSB_BOOTLOADER)
extern void _startup (void);        // See c018i.c in your C18 compiler dir
#pragma code REMAPPED_RESET_VECTOR = REMAPPED_RESET_VECTOR_ADDRESS
void _reset (void)
{
   _asm goto _startup _endasm
}
#endif
#pragma code REMAPPED_HIGH_INTERRUPT_VECTOR = REMAPPED_HIGH_INTERRUPT_VECTOR_ADDRESS
void Remapped_High_ISR (void)
{
    _asm goto YourHighPriorityISRCode _endasm
}
#pragma code REMAPPED_LOW_INTERRUPT_VECTOR = REMAPPED_LOW_INTERRUPT_VECTOR_ADDRESS
void Remapped_Low_ISR (void)
{
```

```
    _asm goto YourLowPriorityISRCode _endasm
}

#if defined(PROGRAMMABLE_WITH_USB_HID_BOOTLOADER)||defined(PROGRAMMABLE_WITH_USB_MCHPUSB_BOOTLOADER)
//Note: If this project is built while one of the bootloaders has
//been defined, but then the output hex file is not programmed with
//the bootloader, addresses 0x08 and 0x18 would end up programmed with 0xFFFF.
//As a result, if an actual interrupt was enabled and occured, the PC would jump
//to 0x08 (or 0x18) and would begin executing "0xFFFF" (unprogrammed space).  This
//executes as nop instructions, but the PC would eventually reach the REMAPPED_RESET_VECTOR_ADDRESS
//(0x1000 or 0x800, depending upon bootloader), and would execute the "goto _startup". This
//would effective reset the application.

//To fix this situation, we should always deliberately place a
//"goto REMAPPED_HIGH_INTERRUPT_VECTOR_ADDRESS" at address 0x08, and a
//"goto REMAPPED_LOW_INTERRUPT_VECTOR_ADDRESS" at address 0x18.  When the output
//hex file of this project is programmed with the bootloader, these sections do not
//get bootloaded (as they overlap the bootloader space).  If the output hex file is not
//programmed using the bootloader, then the below goto instructions do get programmed,
//and the hex file still works like normal.  The below section is only required to fix this
//scenario.
#pragma code HIGH_INTERRUPT_VECTOR = 0x08
void High_ISR (void)
{
    _asm goto REMAPPED_HIGH_INTERRUPT_VECTOR_ADDRESS _endasm
```

```
    }
    #pragma code LOW_INTERRUPT_VECTOR = 0x18
    void Low_ISR (void)
    {
        _asm goto REMAPPED_LOW_INTERRUPT_VECTOR_ADDRESS _endasm
    }
    #endif //end of "#if defined(PROGRAMMABLE_WITH_USB_HID_BOOTLOADER)||defined(PROGRAMMABLE_WITH_USB_LEGACY_CUSTOM_CLASS_BOOTLOADER)"

    #pragma code


    //These are your actual interrupt handling routines.
    #pragma interrupt YourHighPriorityISRCode
    void YourHighPriorityISRCode()
    {
        //Check which interrupt flag caused the interrupt.
        //Service the interrupt
        //Clear the interrupt flag
        //Etc.
#if defined(USB_INTERRUPT)
        USBDeviceTasks();
#endif

    }   //This return will be a "retfie fast", since this is in a #pragma interrupt section
    #pragma interruptlow YourLowPriorityISRCode
    void YourLowPriorityISRCode()
    {
```

```
            //Check which interrupt flag caused the interrupt.
            //Service the interrupt
            //Clear the interrupt flag
            //Etc.
            if (PIR1bits.TX1IF==1)
            {
                inter=0;
                PIE1bits.TX1IE = 0;                 //al entrar en interrupcion quiere decir que ya se ha enviado el dato entonces deshabilitamos la int
                PIR1bits.TX1IF = 0;                 //comprobamos que se ha producido int por transmision y reseteamos la bandera
            }

    }       //This return will be a "retfie", since this is in a #pragma interruptlow section

#elif defined(__C30__)
  #if defined(PROGRAMMABLE_WITH_USB_HID_BOOTLOADER)
    /*
     * ISR JUMP TABLE
     *
     * It is necessary to define jump table as a function because C30 will
     * not store 24-bit wide values in program memory as variables.
     *
     * This function should be stored at an address where the goto instructions
     * line up with the remapped vectors from the bootloader's linker script.
     *
     * For more information about how to remap the interrupt vectors,
     * please refer to AN1157.  An example is provided below for the T2
     * interrupt with a bootloader ending at address 0x1400
```

```
      */
//      void __attribute__ ((address(0x1404))) ISRTable(){
//
//       asm("reset"); //reset instruction to prevent runaway code
//       asm("goto %0"::"i"(&_T2Interrupt));  //T2Interrupt's address
//      }
  #endif
#endif




/** DECLARATIONS ******************************************************/
#pragma code


/*********************************************************************
 * Function:       void main(void)
 *
 * PreCondition:    None
 *
 * Input:          None
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       Main program entry point.
 *
```

```
 * Note:          None
 *******************************************************************************/
#if defined(__18CXX)
void main(void)
#else
int main(void)
#endif
{
   InitializeSystem();
        //OSCCON = 0b01110011;
        //Comenzamos las inicializaciones de nuestro proyecto
        RCONbits.IPEN = 1;                      // habilitacion de prioridades en interrupciones
        TRISB = 0;
        TRISCbits.TRISC7=1;             // habilita El bit para RX
        TRISCbits.TRISC6=0;             // Habilita El bit para TX
        TXSTA=0b00100000;                       // Ver registro
        //RCSTAbits.SPEN=1;                     // SERIAL PORT ENABLE;
        //SPBRG=0xFF;                                   // De acuerdo a la tabla del manual
        UART1Config = USART_TX_INT_ON & USART_RX_INT_OFF & USART_ASYNCH_MODE & USART_EIGHT_BIT & USART_BRGH_HIGH & BAUD_8_BIT_RATE;
        baud = 51;                                      // valor del registro SPBRG para una velocidad de 9600 baudios a 8 MHz
   Open1USART(UART1Config,baud);
        PIE1bits.TX1IE = 0;             //deshabilitamos TX interrupt
         //compare with the table above
   PIR1bits.TX1IF = 0;              //reset TX pin flag
   IPR1bits.TX1IP = 0;              //Not high priority
        INTCON = 0;
        PIE1 = 0;
```

```
        PIE2 = 0;
        PIE3 = 0;
        PIE4 = 0;
        PIE5 = 0;
   INTCONbits.PEIE = 1;                    //Enable pheripheral interrupt (serial port is a pheripheral)
        INTCONbits.GIE = 1;                        //Habilitación global de interrupciones
   //ei();                                 //remember the master switch for interrupt?
        inter=1;                                   //ponemos esta variable a 1 para ayudarnos saber cuando hemos terminado de enviar por USART
        for (i=0;i<=64;i++)
        {
                USB_Out_Buffer[i]=0;
        }

   while(1)
   {
     #if defined(USB_INTERRUPT)
       if(USB_BUS_SENSE && (USBGetDeviceState() == DETACHED_STATE))
       {
          USBDeviceAttach();
       }
     #endif

     #if defined(USB_POLLING)
                // Check bus status and service USB interrupts.
     USBDeviceTasks(); // Interrupt or polling method.  If using polling, must call
                                // this function periodically.  This function will take care
                                // of processing and responding to SETUP transactions
```

```
                        // (such as during the enumeration process when you first
                        // plug in).  USB hosts require that USB devices should accept
                        // and process SETUP packets in a timely fashion.  Therefore,
                        // when using polling, this function should be called
                        // frequently (such as once about every 100 microseconds) at any
                        // time that a SETUP packet might reasonably be expected to
                        // be sent by the host to your device.  In most cases, the
                        // USBDeviceTasks() function does not take very long to
                        // execute (~50 instruction cycles) before it returns.
    #endif


            // Application-specific tasks.
            // Application related code may be added here, or in the ProcessIO() function.

            //Aqui comienza el funcionamiento normal del proyecto
            //Write1USART(0b10000000);              //probando el uart
            //esperaenvio();
            while (USB_Out_Buffer[0]!=15)       //esperamos en este bucle hasta que el PC nos
empiece a enviar los datos
            {
                    USBDeviceTasks();
            ProcessIO();
            }
            //USBDeviceTasks();
            confirma();                                     //le mandamos el codigo al PC
que indica comienzo de transmision
            #if defined(USB_INTERRUPT)
        if(USB_BUS_SENSE && (USBGetDeviceState() == DETACHED_STATE))
```

```
    {
        USBDeviceAttach();
    }
#endif

#if defined(USB_POLLING)
                USBDeviceTasks();
        #endif
        //while(1){
        //      Write1USART(0b11110000);
        //      esperaenvio;
        //}
        Write1USART(0b10000000);        //mandamos a los periféricos la orden de comienzo de programacion
        esperaenvio();                  //mientras todavia hay datos en el buffer de transmision esperamos
        //USBDeviceTasks();
        semaforos=0;
        tfase1=0;
        tfase2=0;
        tfase3=0;
        Delay10KTCYx(200);
        ProcessIO();
        semaforos=USB_Out_Buffer[0];    //el PC nos envia el número de semáforos en primer lugar
        tfase1=USB_Out_Buffer[1];       //hemos recibido del PC la duracion de las fases 1, 2 y 3
        tfase2=USB_Out_Buffer[2];
        tfase3=USB_Out_Buffer[3];
        //USBDeviceTasks();
```

```
            ProcessIO();
            semaforos=USB_Out_Buffer[0];
            tfase1=USB_Out_Buffer[1];
            tfase2=USB_Out_Buffer[2];
            tfase3=USB_Out_Buffer[3];
            disp=0b10000001;
            finconf=0b11000001;
            confirma();
            for (i=1;i<=semaforos;i++)
            {
                  Delay10KTCYx(200);
                  #if defined(USB_INTERRUPT)
      if(USB_BUS_SENSE && (USBGetDeviceState() == DETACHED_STATE))
      {
         USBDeviceAttach();
      }
            #endif

       #if defined(USB_POLLING)
                   USBDeviceTasks();
                  #endif
                  //USBDeviceTasks();
                  ProcessIO();
                  ProcessIO();
                  //USBDeviceTasks();
                  confirma();                    //le decimos al PC que
vaya poniendo en el puerto USB el siguiente paquete
                  Write1USART(disp);             //vamos enviando por USART la
identificacion de disp y sus datos
```

```
                esperaenvio();
                Write1USART(USB_Out_Buffer[0]);
                esperaenvio();
                Write1USART(USB_Out_Buffer[1]);
                esperaenvio();
                Write1USART(USB_Out_Buffer[2]);
                esperaenvio();
                Write1USART(USB_Out_Buffer[3]);
                esperaenvio();
                Write1USART(finconf);          //codigo para indicar fin de conf de
dispositivo
                esperaenvio();
                disp=disp+1;
                finconf=finconf+1;
            }

            Write1USART(0b11100000);           //indicamos a los dispositivos
que se ha terminado la programación del sistema
            esperaenvio();
            Write1USART(0b01000001);           //ponemos todo el sistema en
fase 1
            esperaenvio();
            contfase=0;
            while(1)                           //en este While
escribir el modo de funcionamiento normal despues de tener todo configurado
        {
        Delay10KTCYx(200);                     //retardo de 1 segundo para oscilador
de 8MHz
                Write1USART(0b01100000);       //enviamos el codigo
correspondiente para indicar que ha pasado un segundo
                esperaenvio();
```

```
                    contfase=contfase+1;                    //vamos contando el tiempo
para saber cuando cambiamos de fase
                    if (contfase==tfase1)
                    {
                        Write1USART(0b01000010);            //enviamos la orden de cambio a
fase 2
                        esperaenvio();
                    }
                    if (contfase-tfase1==tfase2)            //enviamos la orden de cambio a fase 3
                    {
                        Write1USART(0b01000011);
                        esperaenvio();
                    }
                    if (contfase-tfase1-tfase2==tfase3)
                    {
                        Write1USART(0b01000001);            //acabada fase 3 volvemos a fase
1
                        esperaenvio();
                        contfase=0;
                    }
                }//end while
    //ProcessIO();
  }//end while
}//end main

void esperaenvio(void)                                      //esta funcion es para esperar
hasta que se haya realizado la transmision por USART
    {
        //PIE1bits.TX1IE = 1;                               //habilitamos la int por
transmision
```

```
                //while (inter==1){}
                while (PIR1bits.TX1IF==0){}
                inter=1;
        }

void ProcessIO(void)
{
   BYTE numBytesRead;

   // User Application USB tasks
   if((USBDeviceState < CONFIGURED_STATE)||(USBSuspendControl==1)) return;
        //comprobamos que el USB esta conectado al PC

   if(USBUSARTIsTxTrfReady())
   {
                //numBytesRead =
                        getsUSBUSART(USB_Out_Buffer,4);         //Ahora tenemos lo que llega del
PC en el vector USB_out_buffer
                        //USB_In_Buffer[0]=15;
                        //putUSBUSART(USB_In_Buffer,1);                 //enviamos el numero 15
al PC para confirmar que hemos recibido y ponga el siguiente paquete en el puerto USB

                /*if(numBytesRead != 0)
                {
                        strcpypgm2ram(buffer,(const rom far char *)"Ha pulsado: ");
                        strncat(buffer,USB_Out_Buffer,numBytesRead);
                        strcatpgm2ram(buffer,(const rom far char *)"\r\n");
                        putsUSBUSART(buffer);
                }*/
        }
```

```
    CDCTxService();
}                   //end ProcessIO


void confirma(void)
{
    BYTE numBytesRead;


    // User Application USB tasks
    if((USBDeviceState < CONFIGURED_STATE)||(USBSuspendControl==1)) return;
        //comprobamos que el USB esta conectado al PC


    if(USBUSARTIsTxTrfReady())
    {
                //numBytesRead =
                    //getsUSBUSART(USB_Out_Buffer,4);
                    USB_In_Buffer[0]=15;
                    putUSBUSART(USB_In_Buffer,1);                   //enviamos el numero 15
al PC para confirmar que hemos recibido y ponga el siguiente paquete en el puerto USB
        }
        CDCTxService();
}



/******************************************************************
 * Function:        static void InitializeSystem(void)
 *
 * PreCondition:    None
 *
 * Input:           None
```

```
 *
 * Output:        None
 *
 * Side Effects:    None
 *
 * Overview:       InitializeSystem is a centralize initialization
 *                 routine. All required USB initialization routines
 *                 are called from here.
 *
 *                 User application initialization routine should
 *                 also be called from here.
 *
 * Note:           None
 *******************************************************************/
static void InitializeSystem(void)
{
  #if (defined(__18CXX) & !defined(PIC18F87J50_PIM))
    ADCON1 |= 0x0F;                 // Default all pins to digital
  #elif defined(__C30__)
        #if defined(__PIC24FJ256DA210__) || defined(__PIC24FJ256GB210__)
            ANSA = 0x0000;
            ANSB = 0x0000;
            ANSC = 0x0000;
            ANSD = 0x0000;
            ANSE = 0x0000;
            ANSF = 0x0000;
            ANSG = 0x0000;
    #else
```

```
        AD1PCFGL = 0xFFFF;
    #endif
  #elif defined(__C32__)
    AD1PCFG = 0xFFFF;
  #endif

  #if defined(PIC18F87J50_PIM) || defined(PIC18F46J50_PIM) || defined(PIC18F_STARTER_KIT_1) || defined(PIC18F47J53_PIM)
        //On the PIC18F87J50 Family of USB microcontrollers, the PLL will not power up and be enabled
        //by default, even if a PLL enabled oscillator configuration is selected (such as HS+PLL).
        //This allows the device to power up at a lower initial operating frequency, which can be
        //advantageous when powered from a source which is not gauranteed to be adequate for 48MHz
        //operation.  On these devices, user firmware needs to manually set the OSCTUNE<PLLEN> bit to
        //power up the PLL.
  {
    unsigned int pll_startup_counter = 600;
    OSCTUNEbits.PLLEN = 1;  //Enable the PLL and wait 2+ms until the PLL locks before enabling USB module
    while(pll_startup_counter--);
  }
  //Device switches over automatically to PLL output after PLL is locked and ready.
  #endif

  #if defined(PIC18F87J50_PIM)
        //Configure all I/O pins to use digital input buffers.  The PIC18F87J50 Family devices
        //use the ANCONx registers to control this, which is different from other devices which
        //use the ADCON1 register for this purpose.
```

```
  WDTCONbits.ADSHR = 1;                 // Select alternate SFR location to access ANCONx registers
  ANCON0 = 0xFF;              // Default all pins to digital
  ANCON1 = 0xFF;              // Default all pins to digital
  WDTCONbits.ADSHR = 0;                 // Select normal SFR locations
  #endif

  #if defined(PIC18F46J50_PIM) || defined(PIC18F_STARTER_KIT_1) || defined(PIC18F47J53_PIM)
      //Configure all I/O pins to use digital input buffers.  The PIC18F87J50 Family devices
      //use the ANCONx registers to control this, which is different from other devices which
      //use the ADCON1 register for this purpose.
  ANCON0 = 0xFF;              // Default all pins to digital
  ANCON1 = 0xFF;              // Default all pins to digital
  #endif

 #if defined(PIC24FJ64GB004_PIM) || defined(PIC24FJ256DA210_DEV_BOARD)
      //On the PIC24FJ64GB004 Family of USB microcontrollers, the PLL will not power up and be enabled
      //by default, even if a PLL enabled oscillator configuration is selected (such as HS+PLL).
      //This allows the device to power up at a lower initial operating frequency, which can be
      //advantageous when powered from a source which is not gauranteed to be adequate for 32MHz
      //operation.  On these devices, user firmware needs to manually set the CLKDIV<PLLEN> bit to
      //power up the PLL.
  {
    unsigned int pll_startup_counter = 600;
    CLKDIVbits.PLLEN = 1;
    while(pll_startup_counter--);
  }
```

```
    //Device switches over automatically to PLL output after PLL is locked and ready.
    #endif


//      The USB specifications require that USB peripheral devices must never source
//      current onto the Vbus pin.  Additionally, USB peripherals should not source
//      current on D+ or D- when the host/hub is not actively powering the Vbus line.
//      When designing a self powered (as opposed to bus powered) USB peripheral
//      device, the firmware should make sure not to turn on the USB module and D+
//      or D- pull up resistor unless Vbus is actively powered.  Therefore, the
//      firmware needs some means to detect when Vbus is being powered by the host.
//      A 5V tolerant I/O pin can be connected to Vbus (through a resistor), and
//      can be used to detect when Vbus is high (host actively powering), or low
//      (host is shut down or otherwise not supplying power).  The USB firmware
//      can then periodically poll this I/O pin to know when it is okay to turn on
//      the USB module/D+/D- pull up resistor.  When designing a purely bus powered
//      peripheral device, it is not possible to source current on D+ or D- when the
//      host is not actively providing power on Vbus. Therefore, implementing this
//      bus sense feature is optional.  This firmware can be made to use this bus
//      sense feature by making sure "USE_USB_BUS_SENSE_IO" has been defined in the
//      HardwareProfile.h file.
    #if defined(USE_USB_BUS_SENSE_IO)
    tris_usb_bus_sense = INPUT_PIN; // See HardwareProfile.h
    #endif

//      If the host PC sends a GetStatus (device) request, the firmware must respond
//      and let the host know if the USB peripheral device is currently bus powered
```

```
//      or self powered.  See chapter 9 in the official USB specifications for details
//      regarding this request.  If the peripheral device is capable of being both
//      self and bus powered, it should not return a hard coded value for this request.
//      Instead, firmware should check if it is currently self or bus powered, and
//      respond accordingly.  If the hardware has been configured like demonstrated
//      on the PICDEM FS USB Demo Board, an I/O pin can be polled to determine the
//      currently selected power source.  On the PICDEM FS USB Demo Board, "RA2"
//      is used for      this purpose.  If using this feature, make sure "USE_SELF_POWER_SENSE_IO"
//      has been defined in HardwareProfile.h, and that an appropriate I/O pin has been mapped
//      to it in HardwareProfile.h.
  #if defined(USE_SELF_POWER_SENSE_IO)
  tris_self_power = INPUT_PIN;      // See HardwareProfile.h
  #endif

  UserInit();

  USBDeviceInit();     //usb_device.c.  Initializes USB module SFRs and firmware
                                     //variables to known states.
}//end InitializeSystem



/********************************************************************
 * Function:       void UserInit(void)
 *
 * PreCondition:    None
 *
 * Input:          None
```

```
 *
 * Output:        None
 *
 * Side Effects:    None
 *
 * Overview:       This routine should take care of all of the demo code
 *                 initialization that is required.
 *
 * Note:
 *
 ************************************************************************/
void UserInit(void)
{
   //Initialize all of the debouncing variables
   buttonCount = 0;
   buttonPressed = FALSE;
   stringPrinted = TRUE;

   //Initialize all of the LED pins
         mInitAllLEDs();

   //Initialize the pushbuttons
   mInitAllSwitches();
}//end UserInit

/********************************************************************
 * Function:       void ProcessIO(void)
 *
```

```
 * PreCondition:    None
 *
 * Input:          None
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       This function is a place holder for other user
 *                 routines. It is a mixture of both USB and
 *                 non-USB tasks.
 *
 * Note:           None
 *******************************************************************/
/*void ProcessIO(void)
{
   BYTE numBytesRead;

   //Blink the LEDs according to the USB device status
   BlinkUSBStatus();
   // User Application USB tasks
   if((USBDeviceState < CONFIGURED_STATE)||(USBSuspendControl==1)) return;

   if(buttonPressed)
   {
      if(stringPrinted == FALSE)
      {
         if(mUSBUSARTIsTxTrfReady())
```

```
        {
            putrsUSBUSART("Button Pressed -- \r\n");
            stringPrinted = TRUE;
        }
    }
}
else
{
    stringPrinted = FALSE;
}

if(mUSBUSARTIsTxTrfReady())
{
                numBytesRead = getsUSBUSART(USB_Out_Buffer,64);
                if(numBytesRead != 0)
                {
                        BYTE i;

                        for(i=0;i<numBytesRead;i++)
                        {
                                switch(USB_Out_Buffer[i])
                                {
                                        case 0x0A:
                                        case 0x0D:
                                                USB_In_Buffer[i] = USB_Out_Buffer[i];
                                                break;
                                        default:
                                                USB_In_Buffer[i] = USB_Out_Buffer[i] + 1;
```

```
                                        break;
                                }


                        }


                        putUSBUSART(USB_In_Buffer,numBytesRead);
                }
        }


  CDCTxService();
}               //end ProcessIO


/********************************************************************
 * Function:       void BlinkUSBStatus(void)
 *
 * PreCondition:    None
 *
 * Input:          None
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       BlinkUSBStatus turns on and off LEDs
 *              corresponding to the USB device state.
 *
 * Note:           mLED macros can be found in HardwareProfile.h
 *              USBDeviceState is declared and updated in
```

```
 *               usb_device.c.
 *******************************************************************/
void BlinkUSBStatus(void)
{
   static WORD led_count=0;

   if(led_count == 0)led_count = 10000U;
   led_count--;

   #define mLED_Both_Off()        {mLED_1_Off();mLED_2_Off();}
   #define mLED_Both_On()         {mLED_1_On();mLED_2_On();}
   #define mLED_Only_1_On()       {mLED_1_On();mLED_2_Off();}
   #define mLED_Only_2_On()       {mLED_1_Off();mLED_2_On();}

   if(USBSuspendControl == 1)
   {
      if(led_count==0)
      {
         mLED_1_Toggle();
         if(mGetLED_1())
         {
            mLED_2_On();
         }
         else
         {
            mLED_2_Off();
         }
      }//end if
```

```
}
else
{
   if(USBDeviceState == DETACHED_STATE)
   {
      mLED_Both_Off();
   }
   else if(USBDeviceState == ATTACHED_STATE)
   {
      mLED_Both_On();
   }
   else if(USBDeviceState == POWERED_STATE)
   {
      mLED_Only_1_On();
   }
   else if(USBDeviceState == DEFAULT_STATE)
   {
      mLED_Only_2_On();
   }
   else if(USBDeviceState == ADDRESS_STATE)
   {
      if(led_count == 0)
      {
         mLED_1_Toggle();
         mLED_2_Off();
      }//end if
   }
   else if(USBDeviceState == CONFIGURED_STATE)
```

```
    {
      if(led_count==0)
      {
        mLED_1_Toggle();
        if(mGetLED_1())
        {
          mLED_2_Off();
        }
        else
        {
          mLED_2_On();
        }
      }//end if
    }//end if(...)
  }//end if(UCONbits.SUSPND...)

}//end BlinkUSBStatus




//
************************************************************************************
**********************

//              **********              USB              Callback              Functions
************************************************************

//
************************************************************************************
**********************

// The USB firmware stack will call the callback functions USBCBxxx() in response to certain USB
related
```

```
// events.  For example, if the host PC is powering down, it will stop sending out Start of Frame (SOF)

// packets to your device.  In response to this, all USB devices are supposed to decrease their power

// consumption from the USB Vbus to <2.5mA each.  The USB module detects this condition (which according

// to the USB specifications is 3+ms of no bus activity/SOF packets) and then calls the USBCBSuspend()

// function.  You should modify these callback functions to take appropriate actions for each of these

// conditions.  For example, in the USBCBSuspend(), you may wish to add code that will decrease power

// consumption from Vbus to <2.5mA (such as by clock switching, turning off LEDs, putting the

// microcontroller to sleep, etc.).  Then, in the USBCBWakeFromSuspend() function, you may then wish to

// add code that undoes the power saving things done in the USBCBSuspend() function.


// The USBCBSendResume() function is special, in that the USB stack will not automatically call this

// function.  This function is meant to be called from the application firmware instead.  See the

// additional comments near the function.


/********************************************************************

 * Function:        void USBCBSuspend(void)

 *

 * PreCondition:    None

 *

 * Input:           None

 *

 * Output:          None

 *

 * Side Effects:    None
```

```
 *
 * Overview:        Call back that is invoked when a USB suspend is detected
 *
 * Note:            None
 *****************************************************************************/
void USBCBSuspend(void)
{
    //Example power saving code.  Insert appropriate code here for the desired
    //application behavior.  If the microcontroller will be put to sleep, a
    //process similar to that shown below may be used:

    //ConfigureIOPinsForLowPower();
    //SaveStateOfAllInterruptEnableBits();
    //DisableAllInterruptEnableBits();
    //EnableOnlyTheInterruptsWhichWillBeUsedToWakeTheMicro(); //should enable at least USBActivityIF as a wake source
    //Sleep();
    //RestoreStateOfAllPreviouslySavedInterruptEnableBits(); //Preferrably, this should be done in the USBCBWakeFromSuspend() function instead.
    //RestoreIOPinsToNormal();
    //Preferrably, this should be done in the USBCBWakeFromSuspend() function instead.

    //IMPORTANT NOTE: Do not clear the USBActivityIF (ACTVIF) bit here.  This bit is
    //cleared inside the usb_device.c file.  Clearing USBActivityIF here will cause
    //things to not work as intended.

  #if defined(__C30__)
    USBSleepOnSuspend();
  #endif
```

```
}

/******************************************************************************
 * Function:        void USBCBWakeFromSuspend(void)
 *
 * PreCondition:    None
 *
 * Input:           None
 *
 * Output:          None
 *
 * Side Effects:    None
 *
 * Overview:        The host may put USB peripheral devices in low power
 *                                  suspend mode (by "sending" 3+ms of idle).  Once in suspend
 *                                  mode, the host may wake the device back up by sending non-
 *                                  idle state signalling.
 *
 *                                  This call back is invoked when a wakeup from USB suspend
 *                                  is detected.
 *
 * Note:            None
 *****************************************************************************/
void USBCBWakeFromSuspend(void)
{
        // If clock switching or other power savings measures were taken when
        // executing the USBCBSuspend() function, now would be a good time to
        // switch back to normal full power run mode conditions.  The host allows
```

```
        // a few milliseconds of wakeup time, after which the device must be
        // fully back to normal, and capable of receiving and processing USB
        // packets.  In order to do this, the USB module must receive proper
        // clocking (IE: 48MHz clock must be available to SIE for full speed USB
        // operation).
}

/********************************************************************
 * Function:       void USBCB_SOF_Handler(void)
 *
 * PreCondition:    None
 *
 * Input:          None
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       The USB host sends out a SOF packet to full-speed
 *                 devices every 1 ms. This interrupt may be useful
 *                 for isochronous pipes. End designers should
 *                 implement callback routine as necessary.
 *
 * Note:           None
 *******************************************************************/
void USBCB_SOF_Handler(void)
{
   // No need to clear UIRbits.SOFIF to 0 here.
```

```
  // Callback caller is already doing that.

  //This is reverse logic since the pushbutton is active low
  if(buttonPressed == sw2)
  {
    if(buttonCount != 0)
    {
      buttonCount--;
    }
    else
    {
      //This is reverse logic since the pushbutton is active low
      buttonPressed = !sw2;

      //Wait 100ms before the next press can be generated
      buttonCount = 100;
    }
  }
  else
  {
    if(buttonCount != 0)
    {
      buttonCount--;
    }
  }
}

/*****************************************************************
```

```
 * Function:        void USBCBErrorHandler(void)
 *
 * PreCondition:    None
 *
 * Input:           None
 *
 * Output:          None
 *
 * Side Effects:    None
 *
 * Overview:        The purpose of this callback is mainly for
 *                  debugging during development. Check UEIR to see
 *                  which error causes the interrupt.
 *
 * Note:            None
 *******************************************************************/
void USBCBErrorHandler(void)
{
   // No need to clear UEIR to 0 here.
   // Callback caller is already doing that.

        // Typically, user firmware does not need to do anything special
        // if a USB error occurs.  For example, if the host sends an OUT
        // packet to your device, but the packet gets corrupted (ex:
        // because of a bad connection, or the user unplugs the
        // USB cable during the transmission) this will typically set
        // one or more USB error interrupt flags.  Nothing specific
        // needs to be done however, since the SIE will automatically
```

```
        // send a "NAK" packet to the host.  In response to this, the
        // host will normally retry to send the packet again, and no
        // data loss occurs.  The system will typically recover
        // automatically, without the need for application firmware
        // intervention.

        // Nevertheless, this callback function is provided, such as
        // for debugging purposes.
}


/********************************************************************
 * Function:       void USBCBCheckOtherReq(void)
 *
 * PreCondition:    None
 *
 * Input:          None
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       When SETUP packets arrive from the host, some
 *                                 firmware must process the request and respond
 *                                 appropriately to fulfill the request.  Some of
 *                                 the SETUP packets will be for standard
 *                                 USB "chapter 9" (as in, fulfilling chapter 9 of
 *                                 the official USB specifications) requests, while
```

```
 *                                  others may be specific to the USB device class
 *                                  that is being implemented.  For example, a HID
 *                                  class device needs to be able to respond to
 *                                  "GET REPORT" type of requests.  This
 *                                  is not a standard USB chapter 9 request, and
 *                                  therefore not handled by usb_device.c.  Instead
 *                                  this request should be handled by class specific
 *                                  firmware, such as that contained in usb_function_hid.c.
 *
 * Note:            None
 *******************************************************************/
void USBCBCheckOtherReq(void)
{
  USBCheckCDCRequest();
}//end


/******************************************************************
 * Function:        void USBCBStdSetDscHandler(void)
 *
 * PreCondition:    None
 *
 * Input:           None
 *
 * Output:          None
 *
 * Side Effects:    None
 *
```

```
 * Overview:        The USBCBStdSetDscHandler() callback function is
 *                                  called when a SETUP, bRequest: SET_DESCRIPTOR request
 *                                  arrives.  Typically SET_DESCRIPTOR requests are
 *                                  not used in most applications, and it is
 *                                  optional to support this type of request.
 *
 * Note:            None
 ********************************************************************/
void USBCBStdSetDscHandler(void)
{
    // Must claim session ownership if supporting this request
}//end



/******************************************************************
 * Function:        void USBCBInitEP(void)
 *
 * PreCondition:    None
 *
 * Input:           None
 *
 * Output:          None
 *
 * Side Effects:    None
 *
 * Overview:        This function is called when the device becomes
 *                  initialized, which occurs after the host sends a
 *                                  SET_CONFIGURATION (wValue not = 0) request.  This
```

```
 *                       callback function should initialize the endpoints
 *                       for the device's usage according to the current
 *                       configuration.
 *
 * Note:         None
 *******************************************************************/
void USBCBInitEP(void)
{
    CDCInitEP();
}

/******************************************************************
 * Function:       void USBCBSendResume(void)
 *
 * PreCondition:    None
 *
 * Input:          None
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       The USB specifications allow some types of USB
 *                       peripheral devices to wake up a host PC (such
 *                       as if it is in a low power suspend to RAM state).
 *                       This can be a very useful feature in some
 *                       USB applications, such as an Infrared remote
 *                       controlreceiver.  If a user presses the "power"
```

```
*                       button on a remote control, it is nice that the
*                       IR receiver can detect this signalling, and then
*                       send a USB "command" to the PC to wake up.
*
*                       The USBCBSendResume() "callback" function is used
*                       to send this special USB signalling which wakes
*                       up the PC.  This function may be called by
*                       application firmware to wake up the PC.  This
*                       function should only be called when:
*
*                       1.  The USB driver used on the host PC supports
*                               the remote wakeup capability.
*                       2.  The USB configuration descriptor indicates
*                               the device is remote wakeup capable in the
*                               bmAttributes field.
*                       3.  The USB host PC is currently sleeping,
*                               and has previously sent your device a SET
*                               FEATURE setup packet which "armed" the
*                               remote wakeup capability.
*
*                       This callback should send a RESUME signal that
*           has the period of 1-15ms.
*
* Note:         Interrupt vs. Polling
*           -Primary clock
*           -Secondary clock ***** MAKE NOTES ABOUT THIS *******
*            > Can switch to primary first by calling USBCBWakeFromSuspend()
```

```
 *              The modifiable section in this routine should be changed
 *              to meet the application needs. Current implementation
 *              temporary blocks other functions from executing for a
 *              period of 1-13 ms depending on the core frequency.
 *
 *              According to USB 2.0 specification section 7.1.7.7,
 *              "The remote wakeup device must hold the resume signaling
 *              for at lest 1 ms but for no more than 15 ms."
 *              The idea here is to use a delay counter loop, using a
 *              common value that would work over a wide range of core
 *              frequencies.
 *              That value selected is 1800. See table below:
 *              ==========================================================
 *              Core Freq(MHz)      MIP         RESUME Signal Period (ms)
 *              ==========================================================
 *                48            12          1.05
 *                 4            1           12.6
 *              ==========================================================
 *              * These timing could be incorrect when using code
 *                optimization or extended instruction mode,
 *                or when having other interrupts enabled.
 *                Make sure to verify using the MPLAB SIM's Stopwatch
 *                and verify the actual signal on an oscilloscope.
 *******************************************************************/
void USBCBSendResume(void)
{
    static WORD delay_count;
```

```
    USBResumeControl = 1;               // Start RESUME signaling

    delay_count = 1800U;                // Set RESUME line for 1-13 ms
    do
    {
        delay_count--;
    }while(delay_count);
    USBResumeControl = 0;
}


/********************************************************************
 * Function:        void USBCBEP0DataReceived(void)
 *
 * PreCondition:    ENABLE_EP0_DATA_RECEIVED_CALLBACK must be
 *                  defined already (in usb_config.h)
 *
 * Input:           None
 *
 * Output:          None
 *
 * Side Effects:    None
 *
 * Overview:        This function is called whenever a EP0 data
 *                  packet is received.  This gives the user (and
 *                  thus the various class examples a way to get
 *                  data that is received via the control endpoint.
 *                  This function needs to be used in conjunction
```

```
 *              with the USBCBCheckOtherReq() function since
 *              the USBCBCheckOtherReq() function is the apps
 *              method for getting the initial control transfer
 *              before the data arrives.
 *
 * Note:          None
 *******************************************************************/
#if defined(ENABLE_EP0_DATA_RECEIVED_CALLBACK)
void USBCBEP0DataReceived(void)
{
}
#endif


/******************************************************************
 * Function:       BOOL USER_USB_CALLBACK_EVENT_HANDLER(
 *                 USB_EVENT event, void *pdata, WORD size)
 *
 * PreCondition:    None
 *
 * Input:          USB_EVENT event - the type of event
 *              void *pdata - pointer to the event data
 *              WORD size - size of the event data
 *
 * Output:         None
 *
 * Side Effects:    None
 *
 * Overview:       This function is called from the USB stack to
```

```
 *              notify a user application that a USB event
 *              occured.  This callback is in interrupt context
 *              when the USB_INTERRUPT option is selected.
 *
 * Note:         None
 *******************************************************************/
BOOL USER_USB_CALLBACK_EVENT_HANDLER(USB_EVENT event, void *pdata, WORD size)
{
    switch(event)
    {
        case EVENT_CONFIGURED:
            USBCBInitEP();
            break;
        case EVENT_SET_DESCRIPTOR:
            USBCBStdSetDscHandler();
            break;
        case EVENT_EP0_REQUEST:
            USBCBCheckOtherReq();
            break;
        case EVENT_SOF:
            USBCB_SOF_Handler();
            break;
        case EVENT_SUSPEND:
            USBCBSuspend();
            break;
        case EVENT_RESUME:
            USBCBWakeFromSuspend();
            break;
```

```
      case EVENT_BUS_ERROR:
        USBCBErrorHandler();
        break;
      case EVENT_TRANSFER:
        Nop();
        break;
      default:
        break;
    }
    return TRUE;
}
```

```
/** EOF main.c ************************************************/
```

9.2 Programación PIC16

9.2.1 Semáforo de 3 colores

```
; ==================================================
; ==                  Programa interno del semáforo        ==
; ==                        Ángel Canelo Caldera        ==
; ==                        MPLAB IDE V.8.92          ==
; ==================================================

#include      <p16f1704.inc>
              ; archivo de descripcion del procesador

                    __CONFIG _CONFIG1, _FOSC_INTOSC & _WDTE_OFF & _PWRTE_OFF & _MCLRE_OFF & _CP_OFF      ; Inicializo el oscilador interno y pongo

                    ; el perro guardian en off
```

```
                __CONFIG _CONFIG2, _WRT_OFF & _LVP_OFF



                cblock 0x20

                palabra, color, rv, vr, amar, disp, program, finconf, finsist, unsegundo, fase,
recibido, pvez  ; poner aqui nuestras variables

                endc


                cblock 0x70
                ;accesible desde cualquier banco

                w_temp, status_temp
                ;para las interrupciones

                endc



;=================================================================

;==                                      PROGRAMA PRINCIPAL
        ==

;=================================================================


;--- Vector de Reset ---

                org             0x0

                goto    inicio                  ; salto al comienzo del programa


;--- Vector interrupciones ---

                org             0x04

                goto    rsi                             ; salto al comienzo de las interrupciones


;--- Programa Principal ---

                org             0x06                    ; comienzo del programa
```

```
inicio          call    ini_es              ; inicializa los puertos del uC
                        call    ini_osc             ; configuramos oscilador interno
                        call    ini_var             ; inicializamos ciertas variables
                        call    ini_serie           ; inicializamos UART
                        call    ini_int             ; configuración interrupciones
                        call    activa_gie          ; habilitación global interrupciones

;--- Programa principal Asignación de los colores a las salidas---
comienzo    movf    finsist,0
            xorlw   b'11100000'    ; cuando termine la conf de semáforos salimos del bucle
            btfss   status,z
            goto    comienzo
            bsf     pvez,0
asignacion          movf    fase,0
            xorlwb'01000001'            ; comprobamos si estamos en fase 1
                        btfss   status,z
                        goto    fase2
                        clrf    fase        ; como ya hemos descubierto que estamos en fase 1 borramos el registro fase para que no se meta otra vez aqui
            btfss color,0       ; vemos que color le corresponde a la fase 1
                        goto    siguiente7
            btfsc   pvez,0      ; la primera vez debemos encender directamente rojo o amarillo según proceda
            goto    siguer
                        btfsc   portc,rc0       ; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
            goto    siguer
            bcf     portc,rc2   ; antes de encender amarilla apagamos verde
            bsf     portc,rc1   ; encendemos luz amarilla
            clrf    unsegundo   ; ponemos a 0 el contador
```

```
espera      movf    amar,0      ; antes de ponerse en rojo esperar tiempo amarillo
            xorwf   unsegundo,0
            btfss   status,z
            goto    espera      ; esperamos hasta que coincidan los valores
            bcf     portc,rc1   ; apagamos luz amarilla antes de poner rojo
siguer      bsf     portc,rc0   ; encendemos luz roja
            bcf     pvez,0      ; ya se ha hecho por primera vez, la siguiente habrá que hacer la comprobación de color de fase anterior
                    goto    fase2
siguiente7  btfsc   pvez,0      ; si es primera vez encendemos verde directamente
            goto    siguev
            btfsc   portc,rc2   ; vemos si en la fase anterior estaba en verde
            goto    siguev
            clrf    unsegundo   ; ponemos a 0 el contador
esperav     movf    rv,0        ; antes de ponerse en verde esperar el desfase rv
            xorwf   unsegundo,0
            btfss   status,z
            goto    esperav     ; esperamos hasta que coincidan los valores
            bcf     portc,rc0   ; apagamos rojo antes de encender verde
siguev      bsf     portc,rc2   ; encendemos luz verde
            bcf     pvez,0      ; ya se ha hecho por primera vez, la siguiente habrá que hacer la comprobación de color de fase anterior
fase2       movf    fase,0
            xorlwb'01000010'    ; comprobamos si estamos en fase 2
                    btfss   status,z
                    goto    fase3
                    clrf    fase        ; como ya hemos descubierto que estamos en fase 2 borramos el registro fase para que no se meta otra vez aqui
            btfss color,1
                    goto    siguiente8
```

```
        btfsc   portc,rc0       ; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
        goto    siguer2
        bcf     portc,rc2       ; apagamos verce antes de encender amarilla
        bsf     portc,rc1       ; encendemos luz amarilla
        clrf    unsegundo       ; ponemos a 0 el contador
espera2 movf    amar,0          ; antes de ponerse en rojo esperar tiempo amarillo
        xorwf   unsegundo,0
        btfss   status,z
        goto    espera2         ; esperamos hasta que coincidan los valores
        bcf     portc,rc1       ; apagamos luz amarilla antes de poner rojo
siguer2         bsf             portc,rc0               ; encendemos luz roja
                goto    fase3
siguiente8      btfsc   portc,rc2       ; vemos si en la fase anterior estaba en verde
        goto    siguev2
        clrf    unsegundo       ; ponemos a 0 el contador
esperav2 movf   rv,0            ; antes de ponerse en verde esperar el desfase rv
        xorwf   unsegundo,0
        btfss   status,z
        goto    esperav2        ; esperamos hasta que coincidan los valores
        bcf     portc,rc0       ; apagamos rojo antes de encender verde
siguev2 bsf             portc,rc2               ; encendemos luz verde
fase3           movf    fase,0
        xorlwb'01000011'        ; comprobamos que estamos en fase 3
                btfss   status,z
                goto    asignacion
                clrf    fase            ; como ya hemos descubierto que estamos en fase 3 borramos el registro fase para que no se meta otra vez aqui
        btfss color,2
```

```
                goto    siguiente9
                btfsc   portc,rc0       ; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
        goto    siguer3
        bcf     portc,rc2       ; apagamos verde antes de encender amarilla
        bsf     portc,rc1       ; encendemos luz amarilla
        clrf    unsegundo       ; ponemos a 0 el contador
espera3 movf    amar,0          ; antes de ponerse en rojo esperar tiempo amarillo
        xorwf   unsegundo,0
        btfss   status,z
        goto    espera3         ; esperamos hasta que coincidan los valores
        bcf     portc,rc1       ; apagamos luz amarilla antes de poner rojo
siguer3 bsf             portc,rc0               ; encendemos luz roja
                goto    asignacion
siguiente9              btfsc   portc,rc2       ; vemos si en la fase anterior estaba en verde
        goto    siguev3
        clrf    unsegundo       ; ponemos a 0 el contador
esperav3 movf   rv,0            ; antes de ponerse en verde esperar el desfase rv
        xorwf   unsegundo,0
        btfss   status,z
        goto    esperav3        ; esperamos hasta que coincidan los valores
        bcf     portc,rc0       ; antes de verde apagamos rojo
siguev3 bsf             portc,rc2               ; encendemos luz verde
        goto  asignacion                ; ha acabado la fase 3 volvemos a la fase1


;=============================================================
```

```
;==                                    Archivos.inc
        ==

;================================================================

#include        <banco.inc>

#include        <inicializaciones.inc>

#include        <rutinas.inc>

#include        <interrupciones.inc>


                        end
```

9.2.2 Semáforo de peatones

```
; ================================================

; ==                    Programa interno del semáforo          ==

; ==                            Ángel Canelo Caldera          ==

; ==                            MPLAB IDE V.8.92              ==

; ================================================


#include        <p16f1704.inc>
                ; archivo de descripcion del procesador


                        __CONFIG _CONFIG1, _FOSC_INTOSC & _WDTE_OFF & _PWRTE_OFF &
_MCLRE_OFF & _CP_OFF      ; Inicializo el oscilador interno y pongo

                                ; el perro guardian en off

                        __CONFIG _CONFIG2, _WRT_OFF & _LVP_OFF



                        cblock 0x20

                        palabra, color, rv, vr, amar, disp, program, finconf, finsist, unsegundo, fase,
recibido, parpadeo, pvez                ; poner aqui nuestras variables

                        endc
```

```
                cblock 0x70
                ;accesible desde cualquier banco

                w_temp, status_temp
                ;para las interrupciones

                endc



;=================================================================

;==                                      PROGRAMA PRINCIPAL
        ==

;=================================================================


;--- Vector de Reset ---

                org             0x0

                goto    inicio                  ; salto al comienzo del programa


;--- Vector interrupciones ---

                org             0x04

                goto    rsi                             ; salto al comienzo de las interrupciones


;--- Programa Principal ---

                org             0x06                    ; comienzo del programa
inicio          call    ini_es                  ; inicializa los puertos del uC

                call    ini_osc                 ; configuramos oscilador interno

                call    ini_var                 ; inicializamos ciertas variables

                call    ini_serie               ; inicializamos UART

                call    ini_int                 ; configuración interrupciones

                call    activa_gie                      ; habilitación global interrupciones
```

```
;--- Programa principal Asignación de los colores a las salidas---
comienzo	movf	finsist,0
		xorlw	b'11100000'	; cuando termine la conf de semáforos salimos del bucle
		btfss	status,z
		goto	comienzo
		bsf	pvez,0
asignacion		movf	fase,0
		xorlwb'01000001'		; comprobamos si estamos en fase 1
				btfss	status,z
				goto	fase2
				clrf	fase		; como ya hemos descubierto que estamos en fase 1 borramos el registro fase para que no se meta otra vez aqui
		btfss	color,0		; vemos que color le corresponde a la fase 1
				goto	siguiente7
				btfsc	pvez,0		; la primera vez debemos encender directamente rojo o amarillo según proceda
		goto	siguer
		btfsc	portc,rc0		; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
		goto	siguer
		bcf	portc,rc2		; antes de encender amarilla apagamos verde
		;bsf	portc,rc1		; encendemos luz amarilla
		clrf	unsegundo		; ponemos a 0 el contador
espera		bsf	parpadeo,0		; indicamos que comienza el parpadeo
		movf	amar,0		; antes de ponerse en rojo esperar tiempo amarillo
		xorwf	unsegundo,0
		btfss	status,z
		goto	espera		; esperamos hasta que coincidan los valores
		bcf	parpadeo,0		; indicamos que ha terminado el parpadeo
		bcf	portc,rc2		; apagamos luz verde antes de encender rojo
```

```
        ;bcf    portc,rc1     ; apagamos luz amarilla antes de poner rojo
siguer      bsf             portc,rc0           ; encendemos luz roja
                            bcf    pvez,0       ; ya se ha hecho por primera vez, la siguiente habrá que hacer la comprobación de color de fase anterior
        goto  fase2
siguiente7          btfsc  pvez,0       ; si es primera vez encendemos verde directamente
        goto   siguev
        btfsc  portc,rc2     ; vemos si en la fase anterior estaba en verde
        goto   siguev
        clrf   unsegundo     ; ponemos a 0 el contador
esperav     movf   rv,0         ; antes de ponerse en verde esperar el desfase rv
        xorwf  unsegundo,0
        btfss  status,z
        goto   esperav       ; esperamos hasta que coincidan los valores
        bcf    portc,rc0     ; apagamos rojo antes de encender verde
siguev      bsf             portc,rc2           ; encendemos luz verde
        bcf     pvez,0          ; ya se ha hecho por primera vez, la siguiente habrá que hacer la comprobación de color de fase anterior
fase2               movf   fase,0
        xorlwb'01000010'    ; comprobamos si estamos en fase 2
                            btfss   status,z
                            goto    fase3
                            clrf    fase        ; como ya hemos descubierto que estamos en fase 2 borramos el registro fase para que no se meta otra vez aqui
        btfss color,1
                            goto    siguiente8
        btfsc   portc,rc0     ; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
        goto    siguer2
        bcf     portc,rc2     ; apagamos verce antes de encender amarilla
```

```
        ;bsf    portc,rc1     ; encendemos luz amarilla
        clrf   unsegundo      ; ponemos a 0 el contador
espera2     bsf    parpadeo,0    ; indicamos que comienza el parpadeo
        movf   amar,0        ; antes de ponerse en rojo esperar tiempo amarillo
        xorwf  unsegundo,0
        btfss  status,z
        goto   espera2       ; esperamos hasta que coincidan los valores
        bcf    parpadeo,0    ; indicamos que ha terminado el parpadeo
        bcf    portc,rc2     ; apagamos luz verde antes de encender rojo
        ;bcf    portc,rc1     ; apagamos luz amarilla antes de poner rojo
siguer2             bsf             portc,rc0               ; encendemos luz roja
                    goto    fase3
siguiente8          btfsc   portc,rc2     ; vemos si en la fase anterior estaba en verde
        goto   siguev2
        clrf   unsegundo      ; ponemos a 0 el contador
esperav2    movf   rv,0          ; antes de ponerse en verde esperar el desfase rv
        xorwf  unsegundo,0
        btfss  status,z
        goto   esperav2       ; esperamos hasta que coincidan los valores
        bcf    portc,rc0     ; apagamos rojo antes de encender verde
siguev2     bsf             portc,rc2               ; encendemos luz verde
fase3               movf    fase,0
        xorlwb'01000011'            ; comprobamos que estamos en fase 3
                    btfss   status,z
                    goto    asignacion
                    clrf    fase          ; como ya hemos descubierto que estamos en fase 3
borramos el registro fase para que no se meta otra vez aqui
        btfss color,2
                    goto    siguiente9
```

```
                    btfsc   portc,rc0        ; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
        goto    siguer3
        bcf     portc,rc2       ; apagamos verde antes de encender amarilla
        ;bsf    portc,rc1       ; encendemos luz amarilla
        clrf    unsegundo       ; ponemos a 0 el contador
espera3 bsf     parpadeo,0      ; indicamos que comienza el parpadeo
        movf    amar,0          ; antes de ponerse en rojo esperar tiempo amarillo
        xorwf   unsegundo,0
        btfss   status,z
        goto    espera3         ; esperamos hasta que coincidan los valores
        bcf     parpadeo,0      ; indicamos que ha terminado el parpadeo
        bcf     portc,rc2       ; apagamos luz verde antes de encender rojo
        ;bcf    portc,rc1       ; apagamos luz amarilla antes de poner rojo
siguer3 bsf                     portc,rc0               ; encendemos luz roja
                                goto    asignacion
siguiente9              btfsc   portc,rc2       ; vemos si en la fase anterior estaba en verde
        goto    siguev3
        clrf    unsegundo       ; ponemos a 0 el contador
esperav3        movf    rv,0            ; antes de ponerse en verde esperar el desfase rv
        xorwf   unsegundo,0
        btfss   status,z
        goto    esperav3        ; esperamos hasta que coincidan los valores
        bcf     portc,rc0       ; antes de verde apagamos rojo
siguev3 bsf                     portc,rc2               ; encendemos luz verde
        goto  asignacion                ; ha acabado la fase 3 volvemos a la fase1
```

```
;==================================================================

;==                                          Archivos.inc
        ==

;==================================================================

#include        <banco.inc>

#include        <inicializaciones.inc>

#include        <rutinas.inc>

#include        <interrupciones.inc>


                        end
```

9.2.3 Semáforo de flecha

```
; ================================================

; ==                    Programa interno del semáforo         ==

; ==                            Ángel Canelo Caldera        ==

; ==                            MPLAB IDE V.8.92            ==

; ================================================


#include        <p16f1704.inc>
                ; archivo de descripcion del procesador


                        __CONFIG _CONFIG1, _FOSC_INTOSC & _WDTE_OFF & _PWRTE_OFF &
_MCLRE_OFF & _CP_OFF      ; Inicializo el oscilador interno y pongo


                                ; el perro guardian en off

                        __CONFIG _CONFIG2, _WRT_OFF & _LVP_OFF



                        cblock 0x20

                        palabra, color, rv, vr, amar, disp, program, finconf, finsist, unsegundo, fase,
recibido, pvez          ; poner aqui nuestras variables
```

```
                endc


                cblock 0x70
                ;accesible desde cualquier banco

                w_temp, status_temp
                ;para las interrupciones

                endc



;=================================================================

;==                                    PROGRAMA PRINCIPAL
        ==

;=================================================================


;--- Vector de Reset ---

                org         0x0

                goto    inicio              ; salto al comienzo del programa


;--- Vector interrupciones ---

                org         0x04

                goto    rsi                         ; salto al comienzo de las interrupciones


;--- Programa Principal ---

                org         0x06                ; comienzo del programa

inicio      call    ini_es              ; inicializa los puertos del uC

                call    ini_osc             ; configuramos oscilador interno

                call    ini_var             ; inicializamos ciertas variables

                call    ini_serie           ; inicializamos UART

                call    ini_int             ; configuración interrupciones

                call    activa_gie                  ; habilitación global interrupciones
```

```
;--- Programa principal Asignación de los colores a las salidas---
comienzo	movf	finsist,0
		xorlw	b'11100000'	; cuando termine la conf de semáforos salimos del bucle
		btfss	status,z
		goto	comienzo
		bsf	pvez,0
asignacion		movf	fase,0
		xorlwb'01000001'		; comprobamos si estamos en fase 1
				btfss	status,z
				goto	fase2
				clrf	fase		; como ya hemos descubierto que estamos en fase 1 borramos el registro fase para que no se meta otra vez aqui
		btfss	color,0		; vemos que color le corresponde a la fase 1
				goto	siguiente7
				btfsc	pvez,0		; la primera vez debemos encender directamente rojo o amarillo según proceda
		goto	siguer
		btfss	portc,rc2		; comprobamos si en la fase anerior estaba en rojo para no encender la luz amarilla
		goto	siguer
		;bcf	portc,rc2		; antes de encender amarilla apagamos verde
		;bsf	portc,rc1		; encendemos luz amarilla
		clrf	unsegundo		; ponemos a 0 el contador
espera		movf	amar,0		; antes de ponerse en rojo esperar tiempo amarillo
		xorwf	unsegundo,0
		btfss	status,z
		goto	espera		; esperamos hasta que coincidan los valores
		;bcf	portc,rc1		; apagamos luz amarilla antes de poner rojo
siguer		bcf		portc,rc2		; sigue apagada la luz verde
```

```
                    bcf     pvez,0          ; ya se ha hecho por primera vez, la siguiente habrá
que hacer la comprobación de color de fase anterior
          goto  fase2
siguiente7          btfsc   pvez,0          ; si es primera vez encendemos verde directamente
          goto    siguev
          btfsc   portc,rc2       ; vemos si en la fase anterior estaba en verde
          goto    siguev
          clrf    unsegundo       ; ponemos a 0 el contador
esperav       movf    rv,0          ; antes de ponerse en verde esperar el desfase rv
          xorwf   unsegundo,0
          btfss   status,z
          goto    esperav         ; esperamos hasta que coincidan los valores
          ;bcf    portc,rc0       ; apagamos rojo antes de encender verde
siguev        bsf             portc,rc2               ; encendemos luz verde
          bcf     pvez,0          ; ya se ha hecho por primera vez, la siguiente habrá que hacer la
comprobación de color de fase anterior
fase2               movf    fase,0
          xorlwb'01000010'    ; comprobamos si estamos en fase 2
                    btfss   status,z
                    goto    fase3
                    clrf    fase          ; como ya hemos descubierto que estamos en fase 2
borramos el registro fase para que no se meta otra vez aqui
          btfss color,1
                    goto    siguiente8
          btfss   portc,rc2       ; comprobamos si en la fase anerior estaba en rojo para no encender
la luz amarilla
          goto    siguer2
          ;bcf    portc,rc2       ; apagamos verce antes de encender amarilla
          ;bsf    portc,rc1       ; encendemos luz amarilla
          clrf    unsegundo       ; ponemos a 0 el contador
```

```
espera2     movf    amar,0      ; antes de ponerse en rojo esperar tiempo amarillo
            xorwf   unsegundo,0
            btfss   status,z
            goto    espera2     ; esperamos hasta que coincidan los valores
            ;bcf    portc,rc1   ; apagamos luz amarilla antes de poner rojo
siguer2             bcf             portc,rc2           ; apagamos verde
                    goto    fase3
siguiente8          btfsc   portc,rc2   ; vemos si en la fase anterior estaba en verde
            goto    siguev2
            clrf    unsegundo   ; ponemos a 0 el contador
esperav2    movf    rv,0        ; antes de ponerse en verde esperar el desfase rv
            xorwf   unsegundo,0
            btfss   status,z
            goto    esperav2    ; esperamos hasta que coincidan los valores
            ;bcf    portc,rc0   ; apagamos rojo antes de encender verde
siguev2     bsf             portc,rc2           ; encendemos luz verde
fase3               movf    fase,0
            xorlwb'01000011'        ; comprobamos que estamos en fase 3
                    btfss   status,z
                    goto    asignacion
                    clrf    fase        ; como ya hemos descubierto que estamos en fase 3
borramos el registro fase para que no se meta otra vez aqui
            btfss color,2
                    goto    siguiente9
                    btfss   portc,rc2       ; comprobamos si en la fase anerior estaba en
rojo para no encender la luz amarilla
            goto    siguer3
            ;bcf    portc,rc2   ; apagamos verde antes de encender amarilla
            ;bsf    portc,rc1   ; encendemos luz amarilla
```

```
        clrf    unsegundo       ; ponemos a 0 el contador
espera3     movf    amar,0      ; antes de ponerse en rojo esperar tiempo amarillo
        xorwf   unsegundo,0
        btfss   status,z
        goto    espera3         ; esperamos hasta que coincidan los valores
        ;bcf    portc,rc1       ; apagamos luz amarilla antes de poner rojo
siguer3     bcf             portc,rc2           ; apagamos verde
                            goto    asignacion
siguiente9          btfsc   portc,rc2       ; vemos si en la fase anterior estaba en verde
        goto    siguev3
        clrf    unsegundo       ; ponemos a 0 el contador
esperav3    movf    rv,0        ; antes de ponerse en verde esperar el desfase rv
        xorwf   unsegundo,0
        btfss   status,z
        goto    esperav3        ; esperamos hasta que coincidan los valores
        ;bcf    portc,rc0       ; antes de verde apagamos rojo
siguev3     bsf             portc,rc2           ; encendemos luz verde
        goto  asignacion            ; ha acabado la fase 3 volvemos a la fase1



;=================================================================
;==                                      Archivos.inc
    ==
;=================================================================
#include     <banco.inc>
#include     <inicializaciones.inc>
#include     <rutinas.inc>
#include     <interrupciones.inc>
```

```
                end
```

9.2.4 Inicializaciones.inc

```
; Inicializaciones


;--- Inicialización de los puertos del uC ---
                ;--- Puerto C ---
ini_es          banco3
                        movlw b'000000'
                        movwf anselc                    ; configuro PortC como E/S digital
                        banco0
                        clrf    portc
        movlw   b'000000'
        movwf   portc
                        banco1
                        movlw b'100000'
                        movwf trisc                     ; configuro puerto b como entrada RB7-5 y RB4-
0 como salida
                        ;movlb   b'11101'
        ;movlw   b'00010100'
        ;movwf   rc4pps
        banco0
                        return


;--- Inizalizar configuración del oscilador ---
ini_osc         banco1
                        movlw b'01110011'               ; oscilador interno a 8MHz
                        movwf osccon
```

```
                banco0
                return

;--- Inicializar interrupciones ---
ini_int         banco1
                clrf    intcon          ; borro registro intcon
                clrf    pie1            ; borro registro pie1 (int por perifericos)
                bsf     intcon,peie     ; habilitamos int por perifericos
                bsf     pie1,rcie       ; int por recepcion en puerto serie
                banco0
                bcf     pir1,rcif
                return


;--- Cofiguracion EUSART ---
ini_serie   banco3
        clrf    baud1con
                movlw b'00110011'       ; para una Fosc de 8 MHz ponemos el valor 51
en SPBRG
                movwf sp1brgl           ; en la parte baja ponemos el 51
                clrf    sp1brgh         ; en la parte alta es 0.
        ;bsf    baud1con,abden
        clrf    tx1sta
        bsf     tx1sta,brgh     ; ponemos el high baudrate a high speed
        bcf     baud1con,brg16  ; elegimos el baudrate de 8 bits
        ;bsf    baud1con,sckp
        bcf     tx1sta,sync     ; habilatamos modo asincrono
        ;bsf    tx1sta,txen
        clrf    rc1sta          ; registro de control de recepción
```

```
        bcf     rc1sta,rx9      ; recepcion de 8 bits
        bsf     rc1sta,spen     ; habilitamos el puerto serie
        bsf     rc1sta,cren     ; habilitamos la recepción
                        ; puede que interese configurar tambien el BAUDCTL
        banco0
        return

;--- Inicializar variables de seguimiento ---
ini_var         banco0
                        clrf    palabra
                        clrf    disp
                        clrf    unsegundo
                        clrf    fase
        clrf    program
                        return

;--- Inicializar GIE ---
activa_gie      bsf             intcon,gie      ; bit de habilitación global de int
                        return
```

9.2.5 Interrupciones.inc

```
; Interrupciones

rsi                     movwf w_temp                    ; W_TEMP = W
                        swapf  status,0                 ; W = STATUS con nibbles intercambiados
                        movwf status_temp               ; STATUS_TEMP = STATUS con nibbles intercambiados
                        ;--------------------------------------------------------------------
        btfss   pir1,rcif
        goto    falsa
```

```
        banco3
        ;btfsc   rc1sta,ferr
        ;goto    falsa
        ;movf    rc1reg,0
        ;movwf   recibido       ; leemos lo que llega por UART
        call     inicial                ; comprobar si es interrupcion por programacion inicial
                        call    dispositivo     ; comprobacion si la interrupcion es por seleccion disp
        call    conf
        call    finpdis
        call    finpsist
        call    unseg
        call    cambiof
falsa     banco0
                        bcf             pir1,rcif               ; antes de acabar reseteamos bandera de int por recepcion
                        ;-------------------------------------------------------------------
                        swapf  status_temp,0; W = STATUS original con nibbles correctos
                        movwf status                    ; STATUS = STATUS original
                        swapf  w_temp,1                 ; W_TEMP = W original con nibbles intercambiados
                        swapf  w_temp,0                 ; W = W original con nibbles correcto
                        retfie

;=============================================================
;==                              Subrutinas Interrupciones                                       ==
;=============================================================
inicial             banco3
        movf    rc1reg,0        ; lo que nos ha llegado por puerto serie lo movemos al acumulador
        xorlw   b'10000000'
```

```
        btfss   status,z
        return
                        banco0
        movlw   b'10000000'
        movwf   program         ; guardamos la orden de programacion inicial para saber que vamos por este paso
        ;call      confdisp
                        ;call     colorfase
                        ;call     amarillo
                        return
;-------------------------------------------------------------
dispositivo banco3
        ;movf    rc1reg
        movf    rc1reg,0
        xorlw   b'10000001'     ; comprobamos que nos llega la direccion del dispositivo
        btfss   status,z
        return
                        banco0
        movlw   b'10000001'
        movwf   disp            ; guardamos la direccion que ha enviado el maestro
        return
;-------------------------------------------------------------
conf       banco0
        movf    disp,0
        xorlw   b'10000001'     ; si anteriormente se ha seleccionado este dispositivo entonces ahora vendrá la informacion
        btfss   status,z
        return
        call    informacion     ; nos vamos a la subrutina para guardar la informacion
```

```
        incf    palabra,1       ; vamos incrementando este registro cada vez que entremos aqui
        return
;-------------------------------------------------------------
finpdis     banco3
        ;movf    rc1reg
        movf    rc1reg,0
        xorlw   b'11000001'     ; comprobamos que ha acabado la configuracion
        btfss   status,z
        return
                        banco0
        movlw   b'11000001'
        movwf   finconf         ; guardamos la información enviada
        clrf    palabra         ; como hemos terminado de configurar el disp borramos este registro que
nos indica la palabra de conf actual
        return
;-------------------------------------------------------------
finpsist    banco3
                        movf    rc1reg,0
        ;movf    recibido,0
        xorlw   b'11100000'     ; comprobamos que ha terminado la programacion del sistema
        btfss   status,z
        return
                        banco0
        movlw   b'11100000'
        movwf   finsist         ; guardamos la informacion enviada
        return
;-------------------------------------------------------------
unseg       banco3
        movf    rc1reg,0
```

```
        ;movf    recibido,0
        xorlw   b'01100000'     ; comprobamos que ha pasado un segundo segun el maestro
        btfss   status,z
        return
                        banco0
        incf    unsegundo,1     ; incrementamos este registro cada vez que llegue este codigo por el
bus
        return
;-------------------------------------------------------------
cambiof     banco3
        movf    rc1reg,0
        xorlw   b'01000001'
        btfss   status,z
        goto    nocero1
        banco0
        movlw   b'01000001'
        movwf   fase
        return
nocero1     movf    rc1reg,0
        xorlw   b'01000010'
        btfss   status,z
        goto    nocero2
        banco0
        movlw   b'01000010'
        movwf   fase
        return
nocero2     movf    rc1reg,0
        xorlw   b'01000011'
        btfss   status,z
```

```
        return
        banco0
        movlw   b'01000011'
        movwf   fase
        return
```

9.2.6 Rutinas.inc

```
; Subrutinas


;--- Duración de la luz amarilla---
;amarillo               escribir retardo adecuado...


;                               return
;--- Guardado de las palabras de configuracion---
informacion banco0
                        movf    palabra,0
        xorlw   b'00000001'     ; comprobamos que es la primera palabra de conf
        btfss   status,z        ; si vamos por la primera entonces la informacion es el color por fase
        goto    desfrv
                        banco3
                        movf    rc1reg,0
                        banco0
        movwf   color           ; guardamos los colores de las fases
        return
desfrv      movf    palabra,0
        xorlw   b'00000010'     ; comprobamos que es la segunda palabra de conf
        btfss   status,z
        goto    desfvr
                        banco3
```

```
        movf    rc1reg,0
                        banco0
        movwf   rv              ; guardamos el desfase de rojo a verde
        return
desfvr      movf    palabra,0
        xorlw   b'00000011'     ; comprobamos que es la tercera palabra de conf
        btfss   status,z
        goto    tamar
                        banco3
        movf    rc1reg,0
                        banco0
        movwf   vr              ; guardamos el desfase de verde a rojo
        return
tamar       movf    palabra,0
        xorlw   b'00000100'
        btfss   status,z
        return              ; se podrian añadir más palabras de configuracion en el futuro
        banco3
                        movf    rc1reg,0
                        banco0
        movwf   amar            ; guardamos el tiempo en amarillo
        clrf    disp            ; ya que está guardada toda la informacion reseteamos este registro para
que no se vuelva a entrar aqui
        return
```

9.2.7 Banco.inc

```
; bancos


;--- banco 0 ---
banco0          macro
```

```
                movlb  b'00000'
                endm

;--- banco 1 ---
banco1          macro
                movlb  b'00001'
                endm

;--- banco 2 ---
banco2          macro
                movlb  b'00010'
                endm

;--- banco 3 ---
banco3          macro
                movlb  b'00011'
                endm
```

9.2.8 Interrupciones.inc en semáforo de peatones

```
; Interrupciones

rsi                     movwf w_temp                    ; W_TEMP = W
                        swapf  status,0             ; W = STATUS con nibbles intercambiados
                        movwf status_temp           ; STATUS_TEMP = STATUS con nibbles intercambiados
                        ;--------------------------------------------------------------------
        btfss   pir1,rcif
        goto    falsa
        banco3
        ;btfsc   rc1sta,ferr
```

```
        ;goto    falsa
        ;movf    rc1reg,0
        ;movwf   recibido       ; leemos lo que llega por UART
        call     inicial                ; comprobar si es interrupcion por programacion inicial
                        call    dispositivo     ; comprobacion si la interrupcion es por seleccion disp
        call    conf
        call    finpdis
        call    finpsist
        call    unseg
        call    cambiof
falsa     banco0
                        bcf             pir1,rcif               ; antes de acabar reseteamos bandera de int por recepcion
                        ;-------------------------------------------------------------------
                        swapf  status_temp,0; W = STATUS original con nibbles correctos
                        movwf status                    ; STATUS = STATUS original
                        swapf  w_temp,1         ;  W_TEMP  =  W  original  con  nibbles intercambiados
                        swapf  w_temp,0         ; W = W original con nibbles correcto
                        retfie

;=============================================================
;==                              Subrutinas Interrupciones                                       ==
;=============================================================
inicial          banco3
        movf    rc1reg,0       ; lo que nos ha llegado por puerto serie lo movemos al acumulador
        xorlw   b'10000000'
        btfss   status,z
        return
```

```
                banco0
        movlw   b'10000000'
        movwf   program         ; guardamos la orden de programacion inicial para saber que vamos por este paso
        ;call    confdisp
                ;call    colorfase
                ;call    amarillo
                return
;-------------------------------------------------------------
dispositivo banco3
        ;movf    rc1reg
        movf    rc1reg,0
        xorlw   b'10000001'    ; comprobamos que nos llega la direccion del dispositivo
        btfss   status,z
        return
                banco0
        movlw   b'10000001'
        movwf   disp          ; guardamos la direccion que ha enviado el maestro
        return
;-------------------------------------------------------------
conf    banco0
        movf    disp,0
        xorlw   b'10000001'    ; si anteriormente se ha seleccionado este dispositivo entonces ahora vendrá la informacion
        btfss   status,z
        return
        call    informacion    ; nos vamos a la subrutina para guardar la informacion
        incf    palabra,1      ; vamos incrementando este registro cada vez que entremos aqui
        return
```

```
;-------------------------------------------------------------
finpdis    banco3
        ;movf    rc1reg
        movf    rc1reg,0
        xorlw   b'11000001'    ; comprobamos que ha acabado la configuracion
        btfss   status,z
        return
                        banco0
        movlw   b'11000001'
        movwf   finconf        ; guardamos la información enviada
        clrf    palabra        ; como hemos terminado de configurar el disp borramos este registro que
nos indica la palabra de conf actual
        return
;-------------------------------------------------------------
finpsist    banco3
                        movf    rc1reg,0
        ;movf    recibido,0
        xorlw   b'11100000'    ; comprobamos que ha terminado la programacion del sistema
        btfss   status,z
        return
                        banco0
        movlw   b'11100000'
        movwf   finsist        ; guardamos la informacion enviada
        return
;-------------------------------------------------------------
unseg      banco3
        movf    rc1reg,0
        ;movf    recibido,0
        xorlw   b'01100000'    ; comprobamos que ha pasado un segundo segun el maestro
```

```
        btfss   status,z
        return
                        banco0
        incf    unsegundo,1     ; incrementamos este registro cada vez que llegue este codigo por el
bus
        btfss   parpadeo,0
        return
        bcf     parpadeo,0
        btfss   portc,rc2
        goto    encender
        bcf     portc,rc2
        return
encender    bsf     portc,rc2
        return
;-------------------------------------------------------------
cambiof     banco3
        movf    rc1reg,0
        xorlw   b'01000001'
        btfss   status,z
        goto    nocero1
        banco0
        movlw   b'01000001'
        movwf   fase
        return
nocero1     movf    rc1reg,0
        xorlw   b'01000010'
        btfss   status,z
        goto    nocero2
        banco0
```

```
        movlw   b'01000010'
        movwf   fase
        return
nocero2     movf    rc1reg,0
        xorlw   b'01000011'
        btfss   status,z
        return
        banco0
        movlw   b'01000011'
        movwf   fase
        return
```

9.3 Programación aplicación de escritorio (Visual Studio 2010)

```
Imports System.Text

Public Class Form1
    Dim ingresar1(79), ingresar2(5), numsemaf(16) As String
    Dim formulario1(3), vectorenvio(80), semaforo1(3), semaforo2(3), semaforo3(3),
semaforo4(3), semaforo5(3), semaforo6(3), semaforo7(3), semaforo8(3), semaforo9(3),
semaforo10(3), semaforo11(3), semaforo12(3), semaforo13(3), semaforo14(3), semaforo15(3),
semaforo16(3), semaforo17(3), semaforo18(3), semaforo19(3) As Byte
    Dim cont, cont2, cont3, envio1, envio2 As Integer
    Dim llenar, codif, llenar2, tsimulacion As Integer
    Dim fin As Boolean
    Private Structure semaforos

        Dim colorf1 As String
        Dim colorf2 As String
        Dim colorf3 As String
        Dim desfaseRV As Byte
        Dim desfaseVR As Byte
        Dim tamarillo As Byte

    End Structure
    Dim formulario2(20) As semaforos
    Dim inbuffer(0) As Byte
    Private Sub cmdcon_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles cmdcon.Click
        If cmdcon.Text = "CONECTAR" Then
            puerto.PortName = cmbsel.SelectedItem
            puerto.Open()
            cmdenviar.Enabled = True
            cmdenviar.Visible = True
            Timer1.Enabled = True
            cmdcon.Text = "DESCONECTAR"
        Else
            If cmdcon.Text = "DESCONECTAR" Then
                Timer1.Enabled = False
                cmdenviar.Enabled = False
```

```
                cmdenviar.Visible = False
                puerto.Close()
                cmdcon.Text = "CONECTAR"
            End If
        End If
    End Sub
    Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles Button1.Click
        ingresar1(76) = "Nº SEMAFOROS"
        ingresar1(77) = "TFASE1"
        ingresar1(78) = "TFASE2"
        ingresar1(79) = "TFASE3"
        For i = 76 To 79
            vectorenvio(i) = 0
        Next

        For i = 76 To 79
            vectorenvio(i) = Val(InputBox(ingresar1(i), "DATOS INICIALES"))
            'boxes(i).Text = formulario1(i)
        Next
    End Sub

    Private Sub config_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles config.Click
        'Dim i As Integer
        ingresar2(0) = "color fase 1"
        ingresar2(1) = "color fase 2"
        ingresar2(2) = "color fase 3"
        ingresar2(3) = "desfase R-V"
        ingresar2(4) = "desfase V-R"
        ingresar2(5) = "tiempo amarillo"
        numsemaf(0) = "CONFIGURACION SEMAFORO 1"
        numsemaf(1) = "CONFIGURACION SEMAFORO 2"
        numsemaf(2) = "CONFIGURACION SEMAFORO 3"
        numsemaf(3) = "CONFIGURACION SEMAFORO 4"
        numsemaf(4) = "CONFIGURACION SEMAFORO 5"
        numsemaf(5) = "CONFIGURACION SEMAFORO 6"
        numsemaf(6) = "CONFIGURACION SEMAFORO 7"
        numsemaf(7) = "CONFIGURACION SEMAFORO 8"
        numsemaf(8) = "CONFIGURACION SEMAFORO 9"
        numsemaf(9) = "CONFIGURACION SEMAFORO 10"
        numsemaf(10) = "CONFIGURACION SEMAFORO 11"
        numsemaf(11) = "CONFIGURACION SEMAFORO 12"
        numsemaf(12) = "CONFIGURACION SEMAFORO 13"
        numsemaf(13) = "CONFIGURACION SEMAFORO 14"
        numsemaf(14) = "CONFIGURACION SEMAFORO 15"
        numsemaf(15) = "CONFIGURACION SEMAFORO 16"
        For i = 0 To vectorenvio(76) - 1
            formulario2(i).colorf1 = ""
            formulario2(i).colorf2 = ""
            formulario2(i).colorf3 = ""
            formulario2(i).desfaseRV = 0
            formulario2(i).desfaseVR = 0
            formulario2(i).tamarillo = 0
        Next
        For i = 0 To 75
            vectorenvio(i) = 0
        Next
        llenar = 0
        codif = 0
        llenar2 = 1
        For i = 0 To vectorenvio(76) - 1
```

```
            formulario2(i).colorf1 = InputBox(ingresar2(0), numsemaf(i))
            'Text2(llenar) = formulario2(cont2).colorf1
            If formulario2(i).colorf1 = "rojo" Then
                vectorenvio(codif) = vectorenvio(codif) + 1
            End If
            llenar = llenar + 1
            formulario2(i).colorf2 = InputBox(ingresar2(1), numsemaf(i))
            'Text2(llenar) = formulario2(cont2).colorf2
            If formulario2(i).colorf2 = "rojo" Then
                vectorenvio(codif) = vectorenvio(codif) + 2
            End If
            llenar = llenar + 1
            formulario2(i).colorf3 = InputBox(ingresar2(2), numsemaf(i))
            'Text2(llenar) = formulario2(cont2).colorf3
            If formulario2(i).colorf3 = "rojo" Then
                vectorenvio(codif) = vectorenvio(codif) + 4
            End If
            llenar = llenar + 1
            formulario2(i).desfaseRV = Val(InputBox(ingresar2(3), numsemaf(i)))
            vectorenvio(llenar2) = formulario2(i).desfaseRV
            'Text2(llenar) = formulario2(cont2).desfaseRV
            llenar = llenar + 1
            llenar2 = llenar2 + 1
            formulario2(i).desfaseVR = Val(InputBox(ingresar2(4), numsemaf(i)))
            vectorenvio(llenar2) = formulario2(i).desfaseVR
            'Text2(llenar) = formulario2(cont2).desfaseVR
            llenar = llenar + 1
            llenar2 = llenar2 + 1
            formulario2(i).tamarillo = Val(InputBox(ingresar2(5), numsemaf(i)))
            vectorenvio(llenar2) = formulario2(i).tamarillo
            'Text2(llenar) = formulario2(cont2).tamarillo
            llenar = llenar + 1
            llenar2 = llenar2 + 2
            codif = codif + 4
        Next
    End Sub

    Private Sub cmdenviar_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles cmdenviar.Click
        Dim outbuffer As Byte() = New Byte(0) {}
        inbuffer(0) = 0
        puerto.DiscardOutBuffer()
        'outbuffer(0) = 15
        vectorenvio(80) = 15
        'For i = 0 To 79
        'vectorenvio(i) = 33      'Damos valores solo para simular el sistema mas rapido que
intruduccion manual
        'Next
        puerto.Write(vectorenvio, 80, 1)
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        'puerto.DiscardOutBuffer()
        'leido.Text = inbuffer(0)
        puerto.Write(vectorenvio, 76, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 0, 4)
        inbuffer(0) = 0
```

```
        'puerto.DiscardOutBuffer()
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 1 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 4, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 2 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 8, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 3 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 12, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 4 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 16, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 5 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 20, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 6 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 24, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
```

```
        If vectorenvio(76) = 7 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 28, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 8 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
        Do While inbuffer(0) <> 15
            puerto.Read(inbuffer, 0, 1)
        Loop
        puerto.Write(vectorenvio, 32, 4)
        inbuffer(0) = 0
        puerto.DiscardInBuffer()
        If vectorenvio(76) = 9 Then
            fintrans.Text = "FINAL DE LA TRANSMISIÓN"
            GoTo Final
        End If
Final:
    End Sub
End Class
```